

Server Installation Guide（服务器安装指南）

目 录

Server Installation Guide (服务器安装指南)	7
目录	7
关于服务器安装指南	10
需求	10
硬件需求	10
软件需求	10
阅读本文档	11
简易安装	11
推荐配置	11
支持的发行版	12
准备工作	12
安装 CentOS	12
安装 EPEL 仓库	12
安装、配置并启动 MySQL	12
安装 BigWorld Technology 服务器包	17
BWMachineD	17
安装 BigWorld Technology 服务器包	17
BigWorld 服务器	17
BigWorld 服务器工具	17
配置 StatLogger	18
配置 MessageLogger	19
重启 StatLogger 和 WebConsole	21
确认工具正在运行	21
连接到 WebConsole	21
自定义安装	22
升级 BigWorld Technology 服务器包	22
升级 BigWorld Technology 服务器包	22

服务器首次运行	23
创建开发者账户	23
账户创建示例	23
常见错误	23
创建新项目	24
特定包的密钥生成	25
LoginApp 密钥对	25
BigWorld 服务器密钥	25
管理 BigWorld 服务器	25
添加新用户	25
访问控制	29
登录 WebConsole	30
启动 BigWorld 服务器	30
集群配置	32
进程配置	32
默认系统启动配置	33
安全	33
CentOS 5	33
CentOS 7	34
保护 WebConsole 安全	34
路由	34
缓冲区大小	35
禁用 cron 作业	35
附录 A. 硬件需求	36
CPU 规格	36
附录 A. 硬件需求	36
双路/单路/四路/刀片	36

网络接口卡	36
磁盘存储	37
电源	37
内存	37
NOC 带宽	38
VMWare	38
附录 B. 安装 CentOS	38
安装	38
安装后设置	40
安装更新	40
配置服务	40
安装构建工具	40
更改 UID	40
附录 C. 将文件复制到 Linux	41
USB 闪存盘 / 外置硬盘	41
安装 NTFS 支持	42
在 Windows 上托管的 Python SimpleHTTPServer	42
在 Windows 上托管的 Python SimpleHTTPServer	42
Windows 网络共享	42
附录 D. 创建自定义 BigWorld 服务器安装	43
自定义 RPM	43
手动安装	43
附录 D. 创建自定义 BigWorld 服务器安装	43
BWMachined	43
服务器	43
服务器工具	44
需求与注意事项	44

附录 E. 理解 BigWorld Machine Daemon (BWMachineD)	45
BWMachined 的工作原理	45
配置 BWMachined	46
附录 E. 理解 BigWorld Machine Daemon (BWMachineD)	46
创建 ~/.bwmachined.conf	46
创建 /etc/bwmachined.conf	47
附录 F. 使用 httpd 设置反向 HTTPS 代理服务器	49
生成 SSL 证书和私钥	49
附录 F. 使用 httpd 设置反向 HTTPS 代理服务器	49
安装 httpd 和所需模块	50
配置 httpd 并将 WebConsole 与其集成	50
配置 httpd 并将 WebConsole 与其集成	50
附录 G. 安装和配置 MongoDB	51
安装 MongoDB	51
配置 MongoDB	51
MongoDB 的推荐 Linux 系统配置	51
附录 G. 安装和配置 MongoDB	52
MongoDB 配置选项	53
MongoDB 分片设置	53
配置 MongoDB 集群	54
用户创建	56
分片初始化	57
Message Logger 配置	57
启用认证	57
使用 MongoDB 备份和恢复日志	58
MongoDB 中 Message Logger 的数据库架构	58
用户日志数据	58
Common Data (公共数据)	60

在 MongoDB 中备份和恢复 Message Logger 日志数据	60
附录 H. 安装和配置 Carbon 与 Graphite	61
Carbon 和 Graphite 前置条件	62
附录 H. 安装和配置	62
Carbon 与 Graphite	62
安装 Carbon / Graphite	62
配置 Carbon	63
配置 WebConsole	64
WebConsole 分析	64
附录 I. 故障排除	65
检查 BWMachined 是否正在运行	65
BWMachined 故障排除	65
StatLogger	66

Server Installation Guide (服务器安装指南)

BigWorld Technology OSE。2014 年 12 月发布。BigWorld Pty Ltd, Level 2, 1 Smail Street Ultimo NSW 2007, Australia www.bigworldtech.com 版权所有 © 2014 BigWorld Pty Ltd。保留所有权利。

目录

关于服务器安装指南 	
3	
需求 	
. 4	
阅读本文档 	
. . 5	
简易安装 	
. 5	
推荐配置 	
. . 6	
准备工作 	
. 7	
安装 BigWorld Technology 服务器包 	
. 16	
自定义安装 	
. 23	
升级 BigWorld Technology 服务器包 	
. 23	
服务器首次运行 	
. 24	
创建开发者账户 	
. 25	
创建新项目 	
. 27	
特定包的密钥生成 	
. 29	
管理 BigWorld 服务器 	
. 29	
登录 WebConsole 	
. 36	

启动 BigWorld 服务器 	36
集群配置 	38
进程配置 	39
安全 	39
路由 	42
缓冲区大小 	43
禁用 cron 作业 	44
附录 A. 硬件需求 	45
CPU 规格 	46
双路/单路/四路/刀片 	47
网络接口卡 	47
磁盘存储 	47
电源 	48
内存 	48
NOC 带宽 	48
VMWare 	49
附录 B. 安装 CentOS 	49
安装 	50

安装后设置 	53
附录 C. 将文件复制到 Linux 	55
USB 闪存盘 / 外置硬盘 	56
在 Windows 上托管的 Python SimpleHTTPServer 	57
Windows 网络共享 	58
附录 D. 创建自定义 BigWorld 服务器安装 	58
自定义 RPM 	59
手动安装 	59
服务器工具 	61
附录 E. 理解 BigWorld Machine Daemon (BWMachineD) 	63
BWMachined 的工作原理 	64
配置 BWMachined 	64
附录 F. 使用 httpd 设置反向 HTTPS 代理服务器 	69
生成 SSL 证书和私钥 	70
安装 httpd 和所需模块 	71
配置 httpd 并将 WebConsole 与其集成 	72
附录 G. 安装和配置 MongoDB 	73
安装 MongoDB 	74

配置 MongoDB 	74
MongoDB 分片设置 	77
启用认证 	83
使用 MongoDB 备份和恢复日志 	84
附录 H. 安装和配置 Carbon 与 Graphite 	88
Carbon 和 Graphite 前置条件 	89
安装 Carbon / Graphite 	90
配置 Carbon 	91
配置 WebConsole 	93
WebConsole 分析 	93
附录 I. 故障排除 	95
检查 BWMachined 是否正在运行 	96
StatLogger 	97
BigWorld 支持 	98

关于服务器安装指南

需求

硬件需求

软件需求

阅读本文档
服务器和工具，以便在最短的时间内看到运行中的环境。本文档假设您已阅读

本文档描述如何快速安装

BigWorld
Server

Overview（服务器概览）并理解 BigWorld 服务器进程之间的基本交互。由于本安装流程大部分是自动化的，这种安装机制可能不适合那些熟悉 BigWorld 并希望自定义安装流程的高级用户。有关更高级的安装文档，请参阅附录 D“创建自定义 BigWorld 服务器安装”。BigWorld 服务器可在大多数主流 PC 桌面硬件上运行，前提是 64 位（x86_64）。有关 BigWorld 服务器的硬件需求和推荐的详细描述，请参阅附录 A“硬件需求”。BigWorld 服务器工具的推荐最低硬件配置如下：1GHz CPU、1GB 内存、30GB 硬盘（用于主操作系统安装）、120GB 硬盘（用于日志）、100Mbit 网卡。如果外部机器为 WebConsole 和 StatLogger 托管 MySQL 服务器，我们建议该机器具有类似的规格。我们还建议，托管 WebConsole 和/或 StatLogger 的机器与 MySQL 服务器之间的网络链接应具有低延迟和高带宽，以获得最佳性能。

阅读本文档

CentOS 5 64 位和 CentOS 7 64 位是开发和生产环境的推荐平台，但也支持 RedHat Enterprise Linux 5 和 RedHat Enterprise Linux 7。有关安装 CentOS 5 的说明，请参阅附录 B“安装 CentOS”。从 RPM 安装软件包时，其他软件依赖项应自动通过 RPM 依赖列表满足。阅读本文档时，我们使用一些细微的约定来区分不同的情况。其中一个约定是区分以 root 用户身份运行的命令与以普通用户账户运行的命令。这种区别与在 Windows 机器上需要管理员访问权限的情况类似。

以 root 用户身份运行的命令将以 # 符号为前缀，而以普通用户身份运行的命令将以 \$ 符号为前缀。

例如，如果我们想以 root 用户身份运行命令 `cat /proc/1/mem`，我们表示为：

```
# cat /proc/1/mem
```

而如果以普通用户身份运行，则表示为：

```
$ cat /proc/1/mem
```

在可能的情况下，我们会明确说明命令应以何种身份运行，但这一视觉指示应该可以在出现混淆时为您提供帮助。我们使用的另一个约定是将命令中需要您根据自身情况或需求替换的部分用斜体表示。

例如，如果我们想使用 `useradd` 命令（后跟一个新用户名），我们表示为：

```
$ useradd <username>
```

这有助于避免本文档中使用的某些复杂或不熟悉的命令中的歧义。

简易安装

推荐配置

准备工作、安装 BigWorld Technology 服务器包、自定义安装、升级 BigWorld Technology 服务器包。安装 BigWorld 服务器包括安装 3 个独立的组件：BigWorld Machine Daemon (BWMachineD)、BigWorld 服务器 (BaseApp、CellApp、DBApp 等)、BigWorld

服务器工具（MessageLogger、WebConsole 等）。BWMachineD 是 BigWorld 服务器和 BigWorld 服务器工具都需要的，但是服务器和服务器工具可以相互独立运行，通常安装在不同的机器上，以避免一个服务的负载问题干扰另一个服务。安装服务器和工具的最简单路径是使用预生成的 RPM 包，这些包已经过测试并已知可在支持的 Linux 发行版上运行。可扩展 BigWorld 服务器环境的推荐系统配置是将 BigWorld 服务器与运行 BigWorld 服务器工具的机器分开。建议将服务器工具隔离在单独的机器上，以确保在任何集群机器上发生高负载情况时，服务器工具执行的日志记录和监控任务的增加不会进一步降低任何活动集群机器的性能。由于在开发和生产环境中日志文件可能会急剧增长，建议在为操作系统安装创建分区时，为 BigWorld 服务器日志创建单独的分区或使用完全独立的硬盘。对于初始安装或小规模开发，将服务器和工具安装在同一台机器上也是完全可以接受的，本安装指南将假设安装是在单台机器上进行的。

支持的发行版

准备工作

安装 CentOS

更新系统软件包

BigWorld 仅正式支持 64 位 CentOS 5.x 和 7.x 发行版，以及 64 位 RedHat Enterprise Linux 5.x 和 7.x 发行版。BigWorld 之前支持过其他 Linux 发行版，但是为了更好地支持客户，我们已整合了支持列表，以便更加集中精力并减少解决客户问题时的变数。CentOS 可以从 CentOS 社区门户下载。RedHat Enterprise Linux 可从 RedHat 网站获得。由于 BigWorld 服务器是一套具有众多依赖项的复杂程序集，因此在继续主要安装之前，确保所有系统已正确安装并做好准备非常重要。以下是安装 BigWorld 服务器和工具所需内容的快速概述：64 位 x86_64 硬件、CentOS 5 64 位或 CentOS 7 64 位（注意，凡提到 CentOS 的地方，均可替换为 Red Hat Enterprise Linux）、启用 EPEL 仓库、安装并配置好 MySQL 并启动。以下各节将更详细地概述如何实现一个可用的 CentOS 安装，为安装 BigWorld 服务器做好准备。关于如何安装 Linux 的指南不在本文档的范围内，因为此过程通常需要根据您的具体情况进行调整。然而，我们提供了 CentOS 安装过程的粗略概述，其中概述了可能对 BigWorld 服务器集群机器有用的特定软件包和配置选项。该概述可在附录 B“安装 CentOS”中找到。任何系统安装后，最好更新所有已安装的系统软件包，因为自您用于安装的安装介质制作以来，可能已有重要的安全修复或其他错误修复，这些修复可能会影响系统的性能和安全性。

安装 EPEL 仓库

安装、配置并启动 MySQL

以 root 身份运行以下命令：

```
# yum update
```

执行此更新后，最好重新启动计算机以应用任何系统更改（如内核更新）。为了在 CentOS 下完全支持服务器工具，需要安装一些默认 CentOS 安装中不可用的软件包。为此，您必须启用由 Fedora 项目提供的 Extra Packages for Enterprise Linux (EPEL) 仓库。Fedora

项目管理此仓库，因为它们是 Red Hat 和 CentOS 分支的基础发行版。要启用 EPEL 仓库，请使用以下命令：



EPEL 软件包不保持与官方发布一样最新。如果您在下载某个版本时遇到问题，请尝试导航到该目录并搜索当前版本的 EPEL 软件包。例如，在 CentOS 5.6 发布时，EPEL 软件包仍然引用 5.4。请根据需要修改以下 URL。以 root 身份运行：

```
# wget http://download.fedoraproject.org/pub/epel/5/x86_64/epel-release-5-4.noarch.rpm
# rpm -Uvh epel-release-5-4.noarch.rpm
```

此步骤现在已使 EPEL 仓库中的软件包可供您的 CentOS 安装使用，将在安装 BigWorld 服务器 RPM 时使用。如果您希望使用 BigWorld WebConsole 和 StatLogger，或使用带 MySQL 支持的 DBApp，则需要此步骤。BigWorld 服务器和服务器工具都可以使用 MySQL 进行持久数据存储。为了利用 MySQL，必须针对每个用例对其进行正确配置。

CentOS 5 上的 MySQL

启动 MySQL 服务器

以下各节中的说明因您安装的 CentOS 版本和 MySQL 实现（Oracle MySQL 或 MariaDB）而异。请根据需要参阅相应的章节。要在 CentOS 5 上安装 MySQL 服务器，请以 root 身份运行以下命令：

```
# yum install mysql-server
```

这只会安装 MySQL 服务器。为了与 MySQL 服务器交互以创建初始数据库和设置用户访问权限，我们需要安装 MySQL 客户端。为此，请以 root 身份运行以下命令：

```
# yum install mysql
```

DBApp 要求其表使用 InnoDB 存储引擎，而 InnoDB 不一定是 MySQL 使用的默认引擎。要使 InnoDB 成为 MySQL 的默认引擎，请编辑文件 `/etc/my.cnf`，并在 `[mysqld]` 节中添加以下条目：

```
default-storage-engine=InnoDB
```

安装 MySQL 服务器后，通常不会运行实际的 MySQL 服务器或配置为重启后重新启动。由于它是大多数环境中的核心组件，在继续之前检查 MySQL 服务器是否设置正确非常重要。以 root 身份运行：

```
# /sbin/chkconfig --levels 345 mysqld on
# /etc/init.d/mysqld start
```

CentOS 7 上的 MySQL

这确保 MySQL 服务器在机器重启后仍会运行，并启动 MySQL 服务器以供立即使用。如果您是 Linux 新手并且不熟悉 `runlevels`（运行级别）的概念，如果您打算维护 BigWorld 服务器集群，了解此概念的基本知识将会很有用。更多信息可在 Red Hat Documentation 网站上找到。



从 CentOS 7 开始，MySQL 已被 MariaDB 替换。MariaDB 是 MySQL 的社区分支，被设计为透明且向后兼容的二进制替换。除非另有说明，BigWorld 文档中的所有 MySQL 引用和命令也适用于 MariaDB。或者，您可以在 CentOS 7 上安装 Oracle MySQL。要在 CentOS 7 上安装 MariaDB 服务器，请以 root 身份运行以下命令：

```
# yum install mariadb-server
```

要在 CentOS 7 上安装 Oracle MySQL 社区版服务器，请以 root 身份运行以下命令：

```
# rpm -Uvh http://dev.mysql.com/get/mysql-community-release-el7-5.noarch.rpm
# yum install mysql-community-server
```

为了与 MariaDB/MySQL 社区版服务器交互以创建初始数据库和设置用户访问权限，我们使用 MariaDB/MySQL 社区版客户端。上述命令也会自动安装 MariaDB/MySQL 社区版客户端。DBApp 要求其表使用 InnoDB 存储引擎，而 InnoDB 不一定是 MariaDB 使用的默认引擎。要使 InnoDB 成为 MariaDB/MySQL 社区版服务器的默认引擎，请编辑文件 `/etc/my.cnf`，并在 `[mysqld]` 节中添加以下条目：

```
default-storage-engine=InnoDB
```

启动 MariaDB/MySQL 社区版服务器

在继续之前，检查 MariaDB/MySQL 社区版服务器是否设置正确非常重要。以 root 身份运行：对于 MariaDB：

```
# systemctl enable mariadb
# systemctl start mariadb
```

对于 MySQL 社区版：

```
# systemctl enable mysqld
# systemctl start mysqld
```

这会启动 MariaDB/MySQL 社区版服务器以供立即使用。它还确保 MariaDB/MySQL 社区版服务器在机器重启后仍会运行。`systemctl enable` 命令使用 `targets`（目标）。如果您不熟悉 `targets` 的概念，如果您打算维护 BigWorld 服务器集群，了解此概念的基本知识将会很有用。更多信息可在 Red Hat Documentation 网站上找到。如果您打算使用带 MySQL 支持的 DBApp，还必须确保已安装 MariaDB/MySQL 客户端开发包，因为重建带 MySQL 支持的 DBApp 需要它。要安装 MariaDB 客户端开发包，请以 root 身份运行以下命令：

```
# yum install mariadb-devel
```

要安装 MySQL 社区版客户端开发包，请以 root 身份运行以下命令：

```
# yum install mysql-community-devel
```

MySQL 账户

BigWorld 服务器工具

BigWorld 服务器和 BigWorld 服务器工具对 MySQL 服务器有不同的需求，因为它们都执行独特的任务。因此，我们建议为 BigWorld

服务器工具和每个将运行 BigWorld 服务器的用户创建单独的账户。默认的 MySQL 安装配置有一个名为 `root` 的用户。这与系统 `root` 用户不同。为了创建新的 MySQL 用户，我们使用 MySQL `root` 用户账户登录 MySQL，如下所示：

```
$ mysql -u root
```

只要 MySQL `root` 用户账户没有设置密码，此命令可以由任何用户运行。



有关 MySQL 账户创建和管理的详细说明，请参阅 MySQL 文档网站，特别是 `Server Administration`（服务器管理）章节的 `MySQL User Account Management`（MySQL 用户账户管理）部分。BigWorld 服务器工具组件 `StatLogger` 需要 MySQL 服务器才能工作，而 `WebConsole` 默认使用 `SQLite`，但如果您愿意，可以配置为使用 MySQL。如果使用 `install_tools.py` 脚本安装或直接从命令行运行，`WebConsole` 默认会使用 MySQL。以下示例说明如何为 `StatLogger` 创建一个 MySQL 用户，用户名为 `bwtools`，密码为 `bwtools_passwd`，从 `localhost` 连接（任何主机使用 `%`），并分配所需的权限以使 `StatLogger` 正常运行。要创建用户并授予权限，请运行以下命令：

```
$ mysql -u root
mysql> GRANT ALL PRIVILEGES ON `bw\_stat\_log\_%\`.* TO 'bwtools'@'localhost' IDENTIFIED BY 'bwtools_passwd';
```

BigWorld 服务器用户账户

以下示例说明如何为 `WebConsole` 创建一个 MySQL 用户，用户名为 `bwtools`，密码为 `bwtools_passwd`，从 `localhost` 连接（任何主机使用 `%`），并分配所需的权限以使 `WebConsole` 使用 `bw_web_console` 数据库正常运行。要创建用户、授予权限并为 `WebConsole` 创建数据库，请运行以下命令：

```
$ mysql -u root
mysql> GRANT ALL PRIVILEGES ON bw_web_console.* TO 'bwtools'@'localhost' IDENTIFIED BY 'bwtools_passwd';
mysql> CREATE DATABASE bw_web_console;
```

创建 MySQL 数据库后，必须将 `WebConsole` 配置为使用 MySQL 而非 `SQLite`。有关如何执行此操作的详细信息，请参阅 `Server Operations Guide`（服务器操作指南）的 `Production Mode vs Development Mode`（生产模式与开发模式）章节。网络中将运行自己的 BigWorld 服务器的每个用户都需要为该服务器实例创建一个 MySQL 账户。例如，在有两个服务器开发者（`Alice` 和 `Bob`）以及一个运行单个服务器的 QA 团队的开发环境中，需要创建三个 MySQL 数据库并分配给每个服务器用户。将数据库用户的权限限制为其需要执行的任务被认为是良好的做法。由于 `DBApp` 进程（及相关工具）所需的权限少于 BigWorld

服务器工具，因此当我们为服务器创建账户时，将更具体地说明该用户拥有的权限。我们创建用于 BigWorld 服务器的数据库时需要三条信息。

- 用户名（在 `bw.xml` 中使用 `<db/mysql/username>` 设置）
- 密码（在 `bw.xml` 中使用 `<db/mysql/password>` 设置）
- 数据库名称（在 `bw.xml` 中使用 `<db/mysql/databaseName>` 设置）

如果您还不熟悉 `bw.xml` 文件，不必担心，本文档后面将更详细地介绍。以下 SQL 命令提供了创建用于 BigWorld 服务器的 MySQL 账户时使用的基本语法。具体示例如下。

用例

```
$ mysql -u root
GRANT SELECT, INSERT, UPDATE, DELETE, ALTER, CREATE, DROP, INDEX ON game_db_name.* TO 'username'@'1
GRANT SELECT, INSERT, UPDATE, DELETE, ALTER, CREATE, DROP, INDEX ON game_db_name.* TO 'username'@'
GRANT RELOAD ON *.* TO 'username'@'localhost' IDENTIFIED BY 'password';
GRANT RELOAD ON *.* TO 'username'@'%' IDENTIFIED BY 'password';
```

上述命令执行以下操作：

- 1 使用 MySQL `root` 账户连接到 MySQL 服务器。默认情况下，使用此账户不需要密码。
- 2 当从本地机器（即 `localhost`）连接到 MySQL 服务器时，为用户 `username` 使用密码 `password` 授予对名为 `game_db_name` 的数据库的 `SELECT`、`INSERT`、`UPDATE`、`DELETE`、`ALTER`、`CREATE` 和 `INDEX` 数据库表操作权限。
- 3 这与上一个 `GRANT` 命令相同，但是此规则用于授予用户从远程机器（即不从 `localhost`）连接到 MySQL 服务器时的访问权限。可以通过在 `@` 符号后指定更严格的模式来限制用户可以从哪些机器连接。
- 4 授予从本地机器对所有数据库的 `RELOAD` 能力的访问权限。
- 5 这与上一个 `GRANT` 相同，但是如前所述，此规则授予从远程机器连接时的权限。要创建游戏数据库，请使用以下命令：

```
$ mysql -u root
mysql> CREATE DATABASE game_db_name;
```

在我们的用例办公室中，有两个服务器开发者，Alice 和 Bob。Alice 参与一个新游戏项目 Parrot Attack 和一个现有游戏项目 Chickens Fight Back 的开发。Bob 则只参与新的 Parrot Attack 游戏的开发。

Alice 需要为她创建两个数据库，每个她正在工作的项目一个，而 Bob 只需要一个数据库用于他自己的开发目的。下表概述了数据库管理员将用于创建所需 MySQL 数据库账户的信息。

数据库名称	用户名	密码
alice_parrot_attack	alice	1234567
alice_chickens_fight_back	alice	1234567
bob_parrot_attack	bob	bobs secret password1

从该表中可以看到，Alice 和 Bob 都为他们正在工作的每个游戏项目分配了自己的数据库。这是为了确保当 Alice 启动自己的 BigWorld 服务器时，她不会影响 Bob 可能正在参与的工作。使用上面的 Bob 示例，我们将为 Bob 创建一个账户，赋予他对自己的 Parrot Attack 数据库的权限，如下所示：

```
$ mysql -u root
mysql> GRANT SELECT, INSERT, UPDATE, DELETE, ALTER, CREATE, DROP, INDEX ON bob_parrot_attack.* TO '
```



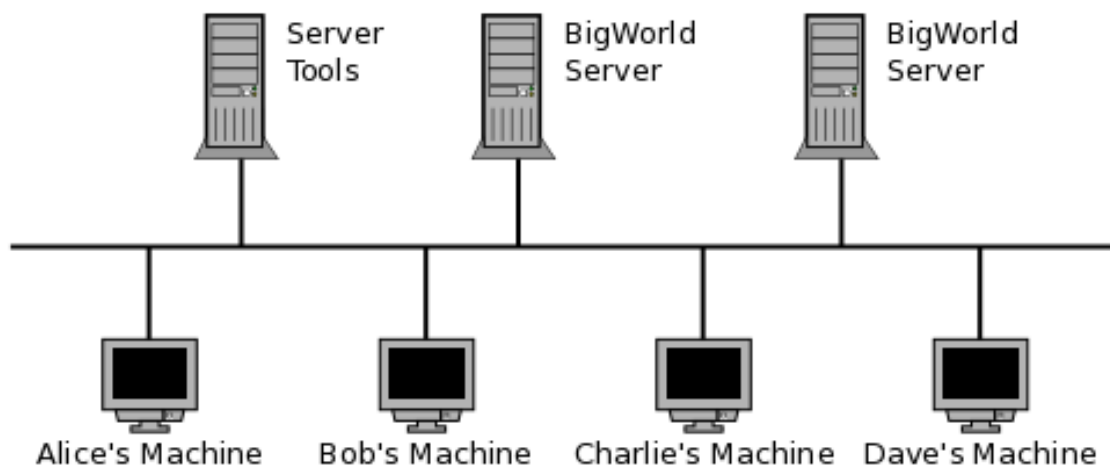
```
Query OK, 0 rows affected (0.08 sec)
mysql> GRANT SELECT, INSERT, UPDATE, DELETE, ALTER, CREATE, DROP, INDEX ON bob_parrot_attack.* TO '
Query OK, 0 rows affected (0.00 sec)
mysql> GRANT RELOAD ON *.* TO 'bob'@'localhost' IDENTIFIED BY 'bobs secret password1';
Query OK, 0 rows affected (0.08 sec)
mysql> GRANT RELOAD ON *.* TO 'bob'@'%' IDENTIFIED BY 'bobs secret password1';
Query OK, 0 rows affected (0.00 sec)
```

安装 BigWorld Technology 服务器包

BWMachineD

安装 BigWorld Technology 服务器包

现在您的系统应该准备好安装下载包中 `rpm` 目录中包含的 RPM 包。本指南假设您已将 RPM 文件复制到 `root` 用户的主目录 `/root`。如果您不确定如何将文件从 Windows 机器传输到新安装的 Linux 机器，请参阅附录 C“将文件复制到 Linux”。这些说明假设您在 Linux 命令行上操作，无论是通过基于文本的登录，还是通过窗口管理器打开的控制台或终端。我们将首先说明基本的办公室安装



BigWorld Machine Daemon 在服务器集群中将运行 BigWorld 进程的所有机器上都是必需的。因此这是我们首先要安装的程序。

```
# cd /root
# yum install --nogpgcheck bigworld-bwmachined-2.1.0.x86_64.rpm
```

我们首先切换到复制 RPM 文件的目录。在本节中，我们假设这是 `/root` 目录（根据需要调整）。然后我们安装 `bwmachined` 包，确保如果版本号不同，请替换为正确的文件名。

BigWorld 服务器

BigWorld 服务器工具

安装服务器工具

从现有的服务器工具安装升级

安装 BigWorld 服务器很简单。以 root 身份运行：

```
# cd /root
# yum install --nogpgcheck bigworld-server-2.1.0.x86_64.rpm
```

虽然 BigWorld 服务器现在应该已成功安装，但在可以运行 BigWorld 服务器实例之前，还剩下两个步骤。第一步需要我们安装 BigWorld 服务器工具，它使您能够启动、停止和与 BigWorld 服务器实例交互。安装服务器工具将在以下章节中更详细地讨论。还需要正确配置用户账户，以便服务器工具能够找到启动 BigWorld 服务器实例时所需的适当可执行文件和游戏资源。这在创建新项目“中简要讨论，并在附录 E“理解 BigWorld Machine Daemon (BWMachineD)”中更详细地讨论。以 root 身份运行以下命令以安装 BigWorld 服务器工具：

```
# cd /root
# yum install --nogpgcheck bigworld-tools-2.1.0.x86_64.rpm
```



从 BigWorld Technology 2.6 开始，bwlockd 实用程序已从 bigworld-tools 包中移除。它必须从 RPM 安装。如果您要从现有的服务器工具安装（从早于 2.9 的 BWT 版本）升级，必须在执行更新之前清除旧的 MessageLogger 数据，因为 MessageLogger 无法使用旧数据。请注意，无法使用 BWT 2.9 服务器工具查看归档的日志数据。要清除旧的 MessageLogger 数据：

配置 StatLogger

配置 MySQL

- 1 停止 Message Logger。
- 2 归档旧的 Message Logger 数据。这可以通过复制或移动整个日志目录来完成。日志目录在 message_logger.conf 中定义为 logdir。或者，您可以使用 mltar.py 实用程序。
- 3 删除日志目录下的所有文件。清除旧的 Message Logger 数据后，按安装服务器工具中所述安装服务器工具。StatLogger 可以与 MySQL 数据库或 Carbon 服务一起工作。以下各节描述如何配置这些版本。注意，Carbon 版本的 StatLogger 仅作为开发者预览提供。在此版本中不完全支持。有关 Carbon 版本的更多信息，请参阅附录 H“安装和配置 Carbon 和 Graphite”。要使用 MySQL 版本的 StatLogger，需要一个 MySQL 账户。从 RPM 安装 StatLogger 时，StatLogger 使用的首选文件（stat_logger.xml）将放置在 /etc/bigworld 目录中。要配置 MySQL，您需要编辑此文件。

要使 StatLogger 与 MySQL 一起工作，您需要将 <database> 下的 <enable> 选项设置为 true，并配置 MySQL 主机、端口、用户名和密码。为了说明如何修改这些值，我们将使用 BigWorld 服务器工具中创建的 MySQL 账户详细信息。使用此信息，我们可以如下设置 stat_logger.xml 的配置选项：

```

<database>
  <enable>true</enable>
  <host>localhost</host>
  <port>3306</port>
  <user>bwtools</user>
  <password>bwtools_passwd</password>
  <prefix>bw_stat_log_data</prefix>
</database>

```

此处的 **prefix** 用于定义用于存储 StatLogger 数据的数据库名称的前缀。

配置 Carbon 支持

配置 MessageLogger



注意，Carbon 版本的 StatLogger 仅作为开发者预览提供。在此版本中不完全支持。在按照以下说明操作之前，请安装和配置 Carbon 和 Graphite，如附录 H“安装和配置 Carbon 和 Graphite”中所述。从 RPM 安装 StatLogger 时，StatLogger 使用的首选文件（**stat_logger.xml**）将放置在 **/etc/bigworld** 目录中。要配置 Carbon 版本的 StatLogger，您需要以 root 身份编辑此文件并更改 **<options>** 中的某些元素。

要使 StatLogger 与 Carbon 一起工作，您需要将 **<carbon>** 的 **<enable>** 设置为 **true**，并配置 Carbon 服务信息。我们将如下设置 **stat_logger.xml** 的配置选项：

```

<carbon>
  <enable>true</enable>
  <host>localhost</host>
  <port>2004</port>
  <prefix>stat_logger</prefix>
</carbon>

```

此处的 **prefix** 声明了 StatLogger 和 StatGrapher 将使用的统计信息的命名空间。要配置 Message Logger：

- 1 选择是使用 MongoDB 还是 MLDB（基于文件的存储）来存储日志数据。使用 MongoDB 的主要优点是可扩展性和按元数据过滤日志消息的能力。MLDB 的优点是数据大小较小，写入日志时性能更好。
- 2 如果要使用 MongoDB，请按照附录 G“安装和配置 MongoDB”中的步骤操作。
- 3 在 **/etc/bigworld/message_logger.conf** 的 **message_logger** 部分设置以下选项：

选项	描述
storage_type	在 mldb 或 mongodb 之间选择（如上面步骤 1 所述）

选项	描述
groups	机器组名称的逗号分隔列表。指定后，MessageLogger 将仅接受来自在 <code>/etc/bwmachined.conf</code> 的 <code>[Groups]</code> 节中列出匹配组名的机器的日志消息。有关更多详细信息，请参阅 Production Scalability 。

- 如果您选择 `mldb` 作为 `storage_type`，请在 `message_logger.conf` 的 `mldb` 部分设置以下选项：

选项	描述
logdir	MessageLogger 将写入其日志的顶级目录的位置。此选项可以是相对路径或绝对路径。如果指定了相对路径，则相对于配置文件的位置计算。
segment_size	日志记录器将自动为特定用户滚动当前日志段的大小（以字节为单位）。
default_archive	<code>mltar.py</code> 使用 <code>--default_archive</code> 选项时使用的文件。此文件也在安装期间插入到 MessageLogger 的 <code>logrotate</code> 脚本中。

- 如果您选择 `mongodb` 作为 `storage_type`，请在 `message_logger.conf` 的 `mongodb` 部分设置以下选项：

选项	描述
host	MongoDB 的主机地址。如果是单实例部署，这可以是 MongoDB 数据库实例的地址；如果是集群部署，则可以是 MongoDB 路由器地址。允许多个 Message Logger 使用相同的 MongoDB 实例，前提是它们具有不同的 <code>loggerID</code> 。有关此多 Message Logger 支持的更多详细信息，请参阅 Database Schema of Message Logger in MongoDB

选项	描述
port	MongoDB 的端口。如果是单实例部署，这可以是 MongoDB 数据库实例的端口；如果是集群部署，则可以是 MongoDB 路由器端口。但是，两种情况的默认端口都是 27017。
user	用于对 MongoDB 进行身份验证的用户名。这是安装和配置 MongoDB 时配置的 MongoDB 服务器用户。

选项	描述
password	上述 MongoDB 用户的密码。
max_buffered_lines	刷新到数据库之前缓冲的最大日志行数。这是为了利用 MongoDB 的批量插入功能来提高写入性能。增加此值将导致缓冲区消耗更多内存，但减少对数据库的写入操作。
flush_interval	刷新日志到 MongoDB 的间隔（以毫秒为单位）。增加此值可能会导致新日志在查询结果中出现得更慢，而减少此值可能会因频繁的写入操作而降低性能。
tcp_time_out	MongoDB 读/写操作的 TCP 超时（以秒为单位）。这仅适用于读写，不适用于连接。连接超时在 MongoDB C++ 驱动程序中固定为 5 秒。
expire_logs_days	日志在从数据库中清除之前保留的天数。过期和清除是针对整个集合的，这意味着只有当集合的所有日志都过期时，该集合才会被过期和清除。包含过期和未过期日志的集合将不会被清除。为了避免当轮换不是每天恰好在同一时间发生时保留额外一天的日志，在检查每个集合的时间戳时将扣除一小时的偏移量。

重启 StatLogger 和 WebConsole

确认工具正在运行

配置 StatLogger 后，您需要启动 StatLogger 并重启 WebConsole，如下所示：在 CentOS 5 上：

```
# /etc/init.d/bw_stat_logger start
| Starting bw_stat_logger: | [ | OK | ]
# /etc/init.d/bw_web_console restart
| Stopping web_console: | [ | OK | ]
| Starting bw_web_console: | [ | OK | ]
```

在 CentOS 7 上：

```
# systemctl restart bw_stat_logger
# systemctl restart bw_web_console
```

在工具机器上安装了服务器工具后，值得确保它们已正确启动，以便您可以确信安装的初始部分没有出现问题。为此，我们只需使用 **status** 命令运行启动脚本，以确保它们按预期工作。以 root 身份运行以下命令：在 CentOS 5 上：

```
# /etc/init.d/bw_stat_logger status
```

连接到 WebConsole

自定义安装

```
Status of stat_logger: running
# /etc/init.d/bw_message_logger status
Status of message_logger: running
# /etc/init.d/bw_web_console status
Status of web_console: running
```

在 CentOS 7 上:

```
# systemctl status bw_stat_logger | grep Active:
Active:active (running) since [...]
# systemctl status bw_message_logger | grep Active:
Active:active (running) since [...]
# systemctl status bw_web_console | grep Active:
Active:active (running) since [...]
```

安装了服务器工具后, 您现在应该能够通过简单地将 Web 浏览器连接到它来看到 WebConsole 页面。WebConsole 的 URL 是它已安装的机器的主机名, 端口为 8080。例如:

```
http://localhost:8080
```

连接后, 您应该会看到类似以下内容的页面:

创建 WebConsole 账户将在服务器首次运行中更详细地讨论。随着开发环境的进展或游戏开始达到其生产周期的发布阶段, 可能需要根据您自己的环境自定义 BigWorld 安装。如果是这种情况, 请参阅附录 D“创建自定义 BigWorld 服务器安装”以获取更多信息。

升级 BigWorld Technology 服务器包

升级 BigWorld Technology 服务器包

当 BigWorld Technology 包发布新版本时, 利用 RPM 的升级功能有助于节省时间并确保安装正确执行。只要您已使用旧的 RPM 包安装, 升级到新包就像使用新 RPM 文件名执行安装操作一样简单。这将自动检测较新的版本并使用新包升级旧包。为此, 请以 root 用户身份执行以下操作:

```
# yum install --nogpgcheck bigworld-package.rpm
```

或者, 如果您利用了 Server Operations Guide 章节中的建议, 使用本地 Yum 仓库安装 RPM, 您可以简单地将新 RPM 文件安装到 Apache 服务器中, 并使用以下命令更新您的主机:

```
# yum update
```

服务器首次运行

创建开发者账户

创建新项目、特定包的密钥生成、管理 BigWorld 服务器、登录 WebConsole、启动 BigWorld 服务器

First Run

本节假设用户完全是 BigWorld 服务器设置的新手，并且在全新安装的机器上工作。因此，我们将逐步介绍一些基本设置步骤，例如为开发者创建一个新的用户账户来运行 BigWorld 服务器。如果您已经对某些步骤感到自信，请随意跳过。以下步骤还将假设操作将使用 Linux 控制台或终端执行。虽然对于选择了 GUI 安装的用户有图形替代方案，但通过描述基于文本的替代方案，我们能够将步骤简化为正在执行的核心行为。



如果您在大型办公环境中工作，您可能希望与系统管理员讨论办公室中已有的用户配置机制。下面描述的方法可能与 LDAP 或 NIS 等其他分布式账户管理系统冲突。每个需要运行自己服务器的开发者都必须为他们创建一个 Linux 用户账户。这使他们能够拥有一个存储其个人开发文件和配置文件的位置，这些配置文件决定了他们的服务器实例将如何启动。以下以 root 用户身份运行的命令用于在 Linux 中创建新的用户账户。

```
# useradd <username>
```

账户创建示例

常见错误

发出此命令后，默认将创建一个新的用户主目录 /home/<username>。创建用户目录后，我们现在需要为用户设置密码，以确保他们是唯一被允许登录的人。为此，以 root 身份发出以下命令：

```
# passwd <username>
Changing password for user <username>.
New UNIX password:
Retype new UNIX password:
passwd: all authentication tokens updated successfully.
```

完成这两个步骤后，您现在有一个新的用户账户，您应该能够使用它登录并继续执行步骤。为了充分说明上述步骤，我们将展示为名为 alice 的新服务器开发者创建新用户账户的整个过程。

```
# useradd alice
# passwd alice
Changing password for user alice.
New UNIX password:
Retype new UNIX password:
passwd: all authentication tokens updated successfully.
```

虽然我们到目前为止还没有详细讨论多计算机安装，但用户账户最常见的错误之一可能是在网络中的不同机器上创建用户账户时使用不同的数字用户 ID (UID)。例如，假设我们有两台服务器机器 **host-A** 和 **host-B**，我们执行以下操作：

```
# On host-A
useradd alice
useradd bob
# On host-B
useradd bob
useradd alice
```

创建新项目

请注意，**host-B** 上的操作顺序与 **host-A** 上执行的操作相反。因此，在 **host-B** 上为 **bob** 用户创建的 UID 可能与 **host-A** 上的 **alice** 用户相同。如果两个用户的 UID 冲突，将会出现服务器管理和服务器监控问题。您可以使用多种替代方法来解决此潜在问题，具体取决于您需要在网络中使用多少台机器。最简单的方法是在创建账户时指定 UID，例如：

```
$ useradd -u 6001 <username>
```

在此示例中，创建用户时指定了 UID 6001，这允许我们在另一台机器上创建账户时指定相同的 UID。然而，当处理大量机器时，这种方法变得不可行，在这种情况下，我们可能考虑使用集中式账户管理解决方案，如 LDAP 或 NIS。这些方法更复杂，超出了本文档的范围。有许多在线资源可以指导您完成设置这些账户管理解决方案的过程。创建用户账户后，您现在可以登录并启动您的第一个项目，这将使您能够启动服务器。一旦您有了以“创建开发者账户”中创建的用户身份登录到新设置机器的终端，创建一个初始示例项目就非常简单。作为服务器安装的一部分，安装了一个名为 **bw_configure** 的程序，用于协助配置您的用户账户以运行 BigWorld 服务器。通过使用新项目名称运行此脚本，它将创建一个新项目并设置我们用于启动服务器的配置文件。要创建新项目，请运行以下命令，将字符串 **<project_name>** 替换为您自己的项目名称：

```
$ bw_configure <project_name>
```

例如，如果服务器开发者 Alice 要创建一个名为 **bigworld_first_run** 的新项目，她将看到以下输出：

```
$ bw_configure bigworld_first_run

'bigworld_first_run' project directory not found. Create 'bigworld_first_run' with tutorial resources
Creating new project at /home/alice/bigworld_first_run
Generating for chapter 6 - BASIC_NPC from /opt/bigworld/2.1/server/bin/res to /home/alice/bigworld_first_run
Writing /home/alice/bigworld_first_run/run.bat
Writing to /home/alice/.bwmachined.conf succeeded
Installation root : /opt/bigworld/current/server
BigWorld resources: /opt/bigworld/current/server/res
| Game resources | : bigworld_first_run |
```

此命令将创建一个目录，其中填充了 BigWorld Tutorial 的资源。它还将在您的主目录中创建一个名为 **.bwmachined.conf** 的新配置文件。作为起点，了解文件的基本分解是有用的，以防您需要在稍后阶段修改它。

`.bwmachined.conf` 文件是一个包含单行的文件，由每台机器上的 `bwmachined` 进程用于确定如何为用户启动服务器进程。此文件中包含的信息必须指向要使用的服务器二进制文件以及游戏资源。该文件的分解如下：

```
<server_binary_directory>;<game_res_directory>[:<secondary_res_directory>]
```

在由 `bw_configure` 程序为您创建的 `.bwmachined.conf` 文件中，上述三个路径将设置如下：`<server_binary_directory>` 会自动填充为 `BigWorld` 服务器二进制文件的安装路径。`<game_res_directory>` 很可能填充为类似于 `/home/<username>/<project_name>/res` 的目录路径。`<project_name>` 目录将是您所有添加和修改发生的地方：它是您的游戏将实现的地方。`<secondary_res_directory>` 会自动填充为 `BigWorld` 资源目录。`BigWorld` 资源目录通常是所有游戏都需要的，因为它包含服务器进程使用的默认配置文件和 Python 库。

有关 `.bwmachined.conf` 文件的更多详细信息，请参阅附录 E“理解 `BigWorld Machine Daemon (BWMachineD)`”。

特定包的密钥生成

LoginApp 密钥对

BigWorld 服务器密钥

管理 BigWorld 服务器

`BigWorld` 服务器包需要生成一些密钥文件并放置在正确的位置才能正常工作。请参阅下面与您的包相关的部分。为了确保客户端连接到真实的游戏服务器，并且客户端发送的用户名/密码不能从公共网络中截获，与 `BigWorld` 服务器到 `LoginApp` 的初始通信使用公钥对加密。`BigWorld Technology` 包附带了一个默认的 `LoginApp` 密钥对，但是由于所有客户都收到相同的密钥对，强烈建议从项目开始就创建自己的密钥对，并将其存储在游戏资源目录中而不是 `BigWorld` 资源目录中。有关如何创建自己的自定义游戏密钥对的信息，请参阅 `Server Programming Guide` 的 `Generating your own RSA keypair` 部分。创建密钥对后，将其放在创建新项目中创建的游戏资源目录中。例如，`loginapp.privkey` 将放在 `/home/<username>/<project_name>/res/server` 中，而 `loginapp.pubkey` 将放在 `/home/<username>/<project_name>/res` 目录中。用户必须将源代码中的 `bigworld.key` 文件放入其游戏资源树中。此文件应放在 `game/res/bigworld/server` 或 `<project_res>/server` 中。创建了项目并设置了所有相关配置文件后，我们可以开始启动服务器。为了与 `BigWorld` 服务器集群交互，我们提供了 `Web` 界面和一组命令行工具。对于大多数服务器交互，我们建议使用 `Web` 界面，我们将在此描述这种方法。`BigWorld` 服务器的 `Web` 界面称为 `WebConsole`。它提供了许多与 `BigWorld` 服务器集群交互的功能，这些功能在 `Server Operations Guide` 的 `Cluster Administration Tools` 章节中有更全面的概述。

添加新用户

添加基于密码的用户

WebConsole 自动启动并运行在安装了 **bigworld-tools** RPM 的机器的端口 8080 上。要连接到 WebConsole，我们将使用类似 **http://<hostname>:8080/** 的 URL，将 **<hostname>** 替换为机器的 IP 地址或主机名，具体取决于您网络中的其他机器是如何配置的。为了避免首次测试 WebConsole 时出现混淆，我们建议使用 WebConsole 机器的 IP 地址，以确认任何问题都与主机名或 DNS 解析问题无关。首次连接时，您应该会看到类似以下内容的屏幕：

WebConsole 登录屏幕 现在需要在 WebConsole 上创建用户账户并更改默认的 **admin** 管理登录密码。为此，使用用户名 **admin**、密码 **admin** 登录。WebConsole 现在支持创建两种不同类型的用户：基于密码的用户和基于 LDAP 的用户。注意，不能同时使用两种类型的用户。使用默认的 WebConsole 设置时，用户是基于密码的。要支持基于 LDAP 的 WebConsole 账户，您需要更改一些 WebConsole 配置选项。请参阅“添加基于密码的用户”以创建基于密码的用户，并参阅“配置 WebConsole 以支持基于 LDAP 的用户”和“添加基于 LDAP 的用户”以配置 WebConsole 和创建基于 LDAP 的用户。使用默认的 WebConsole 设置时，用户是基于密码的。要添加新用户，请单击页面左侧的“Add User”菜单项。将显示以下表单：

下表总结了表单字段：

字段	描述
Username	新用户的用户名。
Password	新用户登录时使用的密码。
Confirm Password	上一字段中重新输入的密码。
Server User	此账户将关联的 Linux 用户。这是 BigWorld 服务器进程将以其身份运行的 Linux 用户。
Group	新用户的用户组。分配的组决定了该用户查看和修改自己的服务器以及他人服务器的访问级别。组权限和访问控制功能在 Cluster Administration Tools 中进一步讨论。

下图显示了正在创建的新用户账户 **AliceB** 并与 Linux 用户账户 **alice** 关联：

配置 WebConsole 以支持基于 LDAP 的用户

Add A New User

Username	AliceB
Password	
Confirm Password	
Server User	alice
Group	modify_all
Add User	

一旦输入了所有用户信息，只需单击 **Add** **User** 按钮，您将返回到主用户列表，其中将包含新用户。

Users			
Username	Server User	Groups	Action
AliceB	alice	modify_all	Action... ▾
admin		admin	Action... ▾

要创建基于 LDAP 的 WebConsole 用户，您首先需要更改 WebConsole 配置文件中的某些配置选项。从命令行运行时，WebConsole 使用 `dev.cfg` 配置文件。作为从 RPM 安装的系统守护程序运行时，WebConsole 使用 `/etc/bigworld/web_console.conf`

配置文件。打开 WebConsole 配置文件后，将身份验证方法设置为 “ldap”，如下所示：

```
identity.auth_method = "ldap"
```

然后在配置文件中配置这些 LDAP 服务器设置。请在设置值之前阅读每个选项上方的注释。以下是一些示例值（将这些值更改为适合您的 LDAP 服务器的适当值）：

```
identity.soldaprovider.host = "127.0.0.1"
identity.soldaprovider.port = 389
identity.soldaprovider.network_time_out = 30
identity.soldaprovider.time_out = 60
```

添加基于 LDAP 的用户

```
identity.soldaprovider.use_tls = "never"
identity.soldaprovider.allow_invalid_tls_cert = True
identity.soldaprovider.use_sasl_digest_md5 = True
identity.soldaprovider.user_dn = "domain_name\\user_name"
identity.soldaprovider.user_password = "password"
identity.soldaprovider.basedn = "DC=domain,DC=com"
identity.soldaprovider.userObjectClass = "person"
identity.soldaprovider.loginUserNameAttr = "sAMAccountName"
identity.soldaprovider.serverUserNameAttr = "sAMAccountName"
```

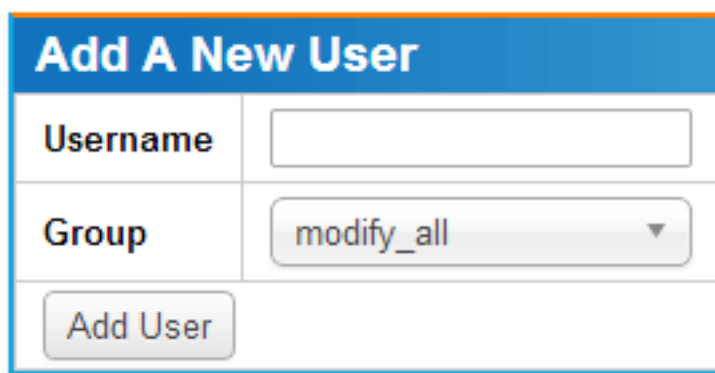
配置 WebConsole 后，按如下方式重启 WebConsole：在 CentOS 5 上：

```
# /etc/init.d/bw_web_console restart
| Stopping web_console: | [ | OK | ]
| Starting bw_web_console: | [ | OK | ]
```

在 CentOS 7 上：

```
# systemctl restart bw_web_console
```

配置 WebConsole 以支持基于 LDAP 的用户后，您现在可以创建基于 LDAP 的用户了。要添加基于 LDAP 的新用户，请单击页面左侧的 “Add User” 菜单项。将显示以下表单：

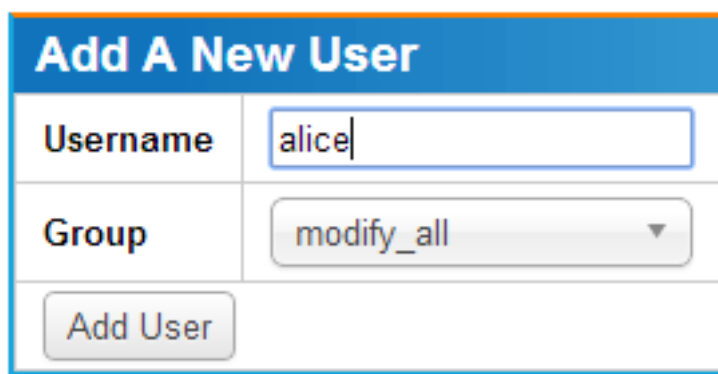


Add A New User	
Username	
Group	<select><option>modify_all</option></select>
Add User	

下表总结了输入字段。

字段	描述
Username	新用户的用户名。此处的用户名必须链接到有效的 LDAP 账户。此字段与 LDAP 账户之间的链接由 WebConsole 的配置项 identity.soldaprovider.loginUserNameAttr 定义。当此用户登录 WebConsole 时，WebConsole 将使用此字段搜索和验证 LDAP。
字段	描述
Group	新用户的用户组。分配的组决定了该用户查看和修改自己的服务器以及他人服务器的访问级别。组权限和访问控制功能在 Server Operations Guide 的 Cluster Administration Tools 中进一步讨论。创建此用户时，WebConsole 将通过 WebConsole 配置项 identity.soldaprovider.serverUserNameAttr 定义的属性从 LDAP 服务器搜索此用户的服务器用户。对于密码，WebConsole 将使用用户登录时输入的密码并针对 LDAP 进行身份验证，因此 WebConsole 不需要在本地存储它。

下图显示了正在创建的新用户账户 **alice**：



一旦输入了用户名并选择了组，单击 **Add User** 按钮。WebConsole 将在 LDAP 服务器上搜索用户信息并创建此用户。如果在此过程中出现错误，将显示错误消息。如果用户创建成功，您将返回到主用户列表，其中将包含新用户：

访问控制

Users			
Username	Server User	Groups	Action
admin		admin	Action... ▼
alice	alice	modify_all	Action... ▼

WebConsole 包括一个灵活的、基于组的访问控制系统。在此系统中，WebConsole 中的每个组都定义了一组权限，这些权限限制了该用户可以查看或修改的范围。默认情况下，仅定义了 2 个权限：**view** 和 **modify**。用户通常以其在用户创建时最初分配的 BigWorld 服务器用户身份与 WebConsole 交互，但是 WebConsole 还提供了临时“充当”不同服务器用户的功能，在此期间所有操作都作为采用的服务器用户执行。因此，2 个权限“view”和“modify”可以应用于 2 种不同的状态：作为自己的服务器用户操作，以及作为其他服务器用户操作，形成 2x2 权限矩阵。默认情况下，WebConsole 预定义了几个组，这些组对应于它们提供的权限，如 WebConsole 的 Admin 模块的“Groups”页面所示。

WebConsole 的 Admin 模块的 Groups 页面，按组显示权限有关自定义权限和组的更多信息，请参阅 Server Programming Guide 的 Customising Access Control 部分。

登录 WebConsole

启动 BigWorld 服务器

现在已创建 WebConsole 用户账户，您可以登录并启动您的第一个 BigWorld 服务器实例。为此，返回 WebConsole 登录屏幕，输入您在上一节中创建的用户名和密码，然后按 Log In 按钮。您现在应该会看到类似以下内容的屏幕：

Cluster 模块的 Processes 页面 页面左侧的每个蓝色条包含一组松散相关的功能，用于与 BigWorld 服务器集群交互。有关每个模块的信息，请参阅屏幕左侧导航菜单中的“Help”菜单选项。经过所有这些艰苦的工作，您现在只需点击几下鼠标就可以启动并运行您的第一个服务器。从 WebConsole Cluster 模块中，单击 Start The Server 按钮。您现在应该会看到一个概述您的用户信息、.bwmachined.conf 文件的各个组件以及可以启动服务器的机器列表的网页：

Server Environment

User	alice
UID	3514
MF_ROOT	/opt/bigworld/current/server
BW_RES_PATH	/home/alice/project_name/res:/opt/bigworld/current/server/res

Start

☒	On a single machine:	centos01-vm ▾
☐	From a saved layout:	▾
☐	On a group of machines:	(use all machines) ▾
	Restrict components by tags:	☒
Go!		

从 Cluster 模块的 Processes 页面启动 BigWorld 服务器 此页面的目的是允许您在启动集群之前查看您的配置设置和您希望在其上启动服务器的机器（或多台机器）。对于我们的目的，我们将使用默认设置，该设置应在单台机器上启动服务器，使用您的主机名或本地机器的 IP 地址标识。要启动服务器，只需按“Go!”按钮。您现在应该会看到一个随着每个服务器进程启动而自动更新的临时网页：

启动服务器时 **Cluster** 模块的 **Processes** 页面视图 上图显示除 **BaseApp** 和 **CellApp** 外的所有服务器进程都已启动并运行。**BaseApp** 和 **CellApp** 进程是最后启动的，因为它们依赖于所有其他服务器进程才能正常运行。您现在应该有一个活动的服务器实例，它向您呈现类似于以下内容的网页：

使用默认布局启动服务器后 **Cluster** 模块的 **Processes** 页面视图 此时，您应该准备好继续 **BigWorld** Tutorial，它将开始引导您完成创建自己的游戏和理解与 **BigWorld** 交互所涉及的编程概念的过程。祝您开发顺利！

集群配置

进程配置

默认系统启动配置

安全、路由、缓冲区大小、禁用 `cron` 作业 网络集群中有许多问题可能会影响 BigWorld 服务器的行为。本章概述了这些问题以及应采取的步骤，以处理这些问题并确保 BigWorld 集群环境的最佳性能。为了帮助系统管理员自定义系统启动脚本的行为（否则不应修改这些脚本），BigWorld 服务器 `init.d` 脚本在启动时从 `/etc/default/<process_name>` 获取每个进程的配置文件。这允许作为一致部署策略的一部分修改脚本使用的变量，而无需修改底层服务脚本。例如，要将 `MessageLogger` 修改为仅接收 `logger` ID 设置为 `"HighPerformance"` 的服务器进程的日志，您可以修改文件 `/etc/default/message_logger` 以包含以下内容：

```
# /etc/default/message_logger
# If set, log only messages destined for this LoggerID.
LOGGER_ID="HighPerformance"
```

目前并非所有进程都有完整的选项集。最佳参考是服务器 `init.d` 脚本。这些脚本位于 CentOS 5 上的 `/etc/init.d` 中，或 CentOS 7 上的 `/opt/bigworld/<version>/tools/init.d` 中。

安全

CentOS 5

在 BigWorld 服务器集群中，并非所有机器都连接到公共互联网，但那些连接到公共互联网的机器必须得到良好的安全保护。最容易实现的方法是使用防火墙阻止传入数据包。通常，对于具有公共 IP 地址的机器，方法应该是在具有公共 IP 的接口上阻止所有传入数据包。例外是 `LoginApp` 和 `BaseApp` 需要允许 UDP 流量到它们监听的端口。要使用的端口在 `res/server/bw.xml` 文件中使用 `loginApp/externalPorts/port` 和 `baseApp/externalPorts/port` 选项定义。有关这些选项的详细信息，请参阅 `Server Operations Guide` 的 `Server Configuration with bw.xml` 章节，sections `BaseApp Configuration Options` 和 `LoginApp Configuration Options`。在 CentOS 5 和 CentOS 7 上配置防火墙的命令不同。它们在下面描述。

使用 Linux 防火墙配置工具 `iptables`，我们可以添加一条规则来丢弃外部接口上的所有传入流量。在以下示例中，我们假设外部接口是 `eth1`。

```
# /sbin/iptables -A INPUT -i eth1 -j DROP
```

在运行 `LoginApp` 的机器上，我们可以添加一条规则以允许登录端口上的流量，使用默认端口 `20013`，如下所示：

```
# /sbin/iptables -I INPUT 1 -p udp -i eth1 --destination-port 20013 -j ACCEPT
```

对于运行 `BaseApp` 的机器，我们添加类似的规则以允许 `baseApp/externalPorts/port` 选项中指定的 `BaseApp` 外部端口上的流量。

我们在新规则中使用 `-I INPUT 1` 而不是 `-A INPUT`，因为 `iptables` 将链中匹配传入数据包的第一条规则应用于数据包。因此，我们需要在拒绝 `eth1` 上所有 UDP 流量的规则之前插入接受登录数据包的规则。对于生产服务器，您应该禁用除来自受信任 IP 地址的 SSH 之外的所有网络服务。

CentOS 7

BWMachined 需要在内部接口上广播并在 UDP 端口 20018 和 20019 上接收自己的回复的能力。此处内部接口表示为 `eth0`。防火墙规则应满足此要求。例如：

```
# /sbin/iptables -I INPUT 1 -p udp -i eth0 -m multiport --destination-ports 20018:20019 -j ACCEPT
```

有关 BigWorld 服务器使用的端口的详细信息，请参阅 Server Security。您必须确保每次机器启动时都恢复防火墙规则。您可以使用以下命令保存配置：

```
# /etc/init.d/iptables save
```

在 CentOS 7 上，默认情况下防火墙由 `firewalld` 控制，并提供了 `firewall-cmd` 命令来管理其策略。`firewalld`

提供基于区域的策略，因此配置也将是特定于区域的。您可以运行此命令获取活动区域：

```
# firewall-cmd --get-active-zones
public
```

假设活动区域是 `public`，此处的示例基于此假设。如果您使用不同的区域，则在运行命令时需要指定匹配的区域名称。`public` 区域的默认设置拒绝除 SSH 服务之外的所有传入连接。在运行 LoginApp 的机器上，我们可以添加一条规则以允许登录端口上的流量，使用默认端口 20013，如下所示：

```
# firewall-cmd --permanent --zone=public --add-port=20013/udp --permanent
```

我们在新规则中使用 `--permanent` 使规则持久化，即使在服务器重启后也保持。

保护 WebConsole 安全

路由

对于运行 BaseApp 的机器，我们添加类似的规则以允许 `baseApp/externalPorts/port` 选项中指定的 BaseApp 外部端口上的流量。对于生产服务器，您应该禁用除来自受信任 IP 地址的 SSH 之外的所有网络服务。BWMachined 需要在内部接口上广播并在 UDP 端口 20018 和 20019 上接收自己的回复的能力。防火墙规则应满足此要求。例如：

```
# firewall-cmd --permanent --zone=public --add-port=20018-20019/udp
```

有关 BigWorld 服务器使用的端口的详细信息，请参阅 Server Security。您可以使用以下命令重新加载新配置以使其在更改后立即生效：

```
# firewall-cmd --reload
```

要使 WebConsole 更安全，您可以将其隐藏在反向 HTTPS 代理服务器后面。代理服务器使用 HTTPS 与客户端安全通信。您可以使用现有的 HTTPS 代理服务器或设置一个新的。有多种免费开源软件包可用于设置反向 HTTPS 代理服务器，例如 Apache HTTP Server (“httpd”) 和 nginx。请参阅附录 F“使用 httpd 设置反向 HTTPS 代理服务器”以获取有关如何使用 Apache HTTP Server 设置反向 HTTPS 代理服务器并将其与 WebConsole 集成的信息。BigWorld Technology 中的大量工具和服务器组件依赖于能够向默认广播地址 (255.255.255.255) 发送 IP 广播数据包

，并且这些数据包能够被正确路由。这在只有一张网络接口的机器上（即仅在内部网络上的机器，如 CellApp 机器、DBApp 机器等）将默认发生。

缓冲区大小

对于具有两张网络接口的机器（即 `BaseApp` 和 `LoginApp` 机器），我们需要确保发送到广播地址的数据包通过内部接口路由。我们可以通过使用 `ip` 命令在内核路由表中创建条目来确保正确完成此操作。此命令可能默认未安装。您可以通过以 `root` 身份运行以下命令来安装此实用程序：

```
# yum install iproute
```

在下面的示例中，我们再次假设接口 `eth0` 是内部网络。要添加默认广播路由，请以 `root` 用户身份运行以下命令：

```
# /sbin/ip route add broadcast 255.255.255.255 dev eth0
```

此命令只会将路由添加到当前路由表，不会在重启机器后应用。为了确保每当 `eth0` 接口联机时都应用此路由，请以 `root` 用户身份运行以下命令：

```
# echo "broadcast 255.255.255.255 dev eth0" > /etc/sysconfig/network-scripts/route-eth0
```

此命令将创建 `/etc/sysconfig/network-scripts/route-eth0` 文件（如果它不存在）。BigWorld 的某些网络组件需要通常大于系统默认值的套接字缓冲区。为了使这些组件正常工作，必须增加为这些缓冲区分配的内存量。这涉及以下值：最大读写缓冲区大小，以及默认写缓冲区大小。如果您使用 `RPM` 包安装了 `BWMachined`，这些值应该已自动添加或更新到 `/etc/sysctl.conf` 或 `/etc/sysctl.d/bwmachined2.conf`（7.x 发行版）文件中，您可以跳过本节。

禁用 cron 作业

为套接字缓冲区分配的大小由内核设置决定，可以使用 `sysctl` 命令动态修改。例如，可以使用以下命令将读缓冲区的最大大小增加到 16 MB：

```
# /sbin/sysctl -w net.core.rmem_max=16777215
```

但是，要使这些更改持久化，我们强烈建议在 `/etc/sysctl.conf` 或 `/etc/sysctl.d/bwmachined2.conf` 文件中定义更高的值。相关设置的条目应具有以下值：

```
net.core.rmem_max = 16777216
net.core.wmem_max = 1048576
net.core.wmem_default = 1048576
```

`Cron` 是一个系统守护程序，它使任务能够按预定间隔运行，例如每小时、每天、每周等。`Cron` 将这些周期性任务称为 `jobs`（作业）。由于这些作业执行的功能类型，这些作业可能会对运行中的服务器的性能产生不利影响。例如，`cron` 作业更新 `locate` 数据库（涉及在整个机器上执行递归目录列表）非常常见。这些类型的作业可能涉及从硬盘的每个部分读取，实际上导致 `Linux` 在内存中的磁盘缓存刷新。这可能会导致服务器进程的性能瞬间下降，因为主机上开始发生磁盘交换。我们建议 BigWorld 服务器机器在生产环境中禁用资源密集型（CPU、内存或磁盘）的 `cron` 作业。您可以通过禁用这些

作业来实现此目的（具有不同的粒度级别）。可以通过清除相关作业的可执行位来禁用 `cron` 作业。例如，要禁任由 `/etc/cron.daily/makewhatis.cron` 运行的作业：

```
# chmod -x /etc/cron.daily/makewhatis.cron
```

可以使用反向操作设置可执行位来重新启用 `cron` 作业：

```
# chmod +x /etc/cron.daily/makewhatis.cron
```

系统 `cron` 作业存储在以下位置：

- `/etc/cron.d`（包含系统服务的 `cron` 作业）
- `/etc/cron.hourly`（用于每小时 `cron` 作业脚本）
- `/etc/cron.daily`（用于每天 `cron` 作业脚本）
- `/etc/cron.weekly`（用于每周 `cron` 作业脚本）
- `/etc/cron.monthly`（用于每月 `cron` 作业脚本）

还要删除任何不必要的用户级 `cron` 作业。可以使用以下命令按用户列出这些作业：

```
$ crontab -l
```



我们不建议完全禁用 `cron` 服务，因为日志轮换和某些安全机制等设施可能依赖于 `cron` 服务处于活动状态。

附录 A. 硬件需求

CPU 规格

附录 A. 硬件需求

双路/单路/四路/刀片、网络接口卡、磁盘存储、电源、内存、NOC 带宽、VMWare
本附录旨在概述运行 BigWorld 所需的硬件规格。以下列表应被视为所需硬件的最低要求：1GHz CPU（首选非移动 CPU）、256MB 内存、8GB 硬盘、100Mbps 网络接口卡。以下列表概述了 BigWorld 推荐的硬件要求，以获得最佳性价比解决方案。通常，使用您可以购买的最快的 CPU。更快的 CPU 意味着每个 CPU 上有更多实体和更少的机器。至于购买哪种 CPU，请查看服务器通常关注的事项：大的 L1 和 L2 缓存大小以及快速的前端总线总是比小的 L1 和 L2 缓存大小和慢的前端总线更好。由于网络流量更少和机器更少，多处理器将帮助您，直到处理器生成的数据过多而无法到达网卡（或通过 PCI 总线到达网卡）。如果您有多个处理器，请确保每个 CPU 上运行一个服务器组件。

双路/单路/四路/刀片

网络接口卡

通常，每个机箱中的 CPU 密度是一个价格决策。刀片机器价格昂贵，但如果您为 NOC 支付昂贵的费率，它可能更便宜。如果我们忽略 NOC 成本，我们推荐双 CPU 机器。一个示例刀片设置如下：

BladeCenter LS20 885051U

- 处理器：低功耗 AMD Opteron 处理器型号 246（标准）
- 内存：4 GB PC3200 ECC DDR RDIMM（2 x 2 GB 套件）系统内存
- IBM eServer BladeCenter™ 千兆以太网扩展卡
- 硬盘驱动器 1：73GB 非热插拔 2.5" 10K RPM Ultra320 SCSI HDD

BladeCenter 86773XU

- 光学设备：IBM 8X Max DVD-ROM Ultrabay Slim Drive（标准）
- 软盘驱动器：IBM 1.44MB 3.5-inch Diskette Drive（标准）
- 电源模块 1 和 2：BladeCenter 2000W 电源一和二（标准）
- 管理模块：BladeCenter KVM / 管理模块（标准）
- 交换模块托架 1：IBM eServer BladeCenter 的 Nortel Networks Layer 2/3 Copper GbE Switch Module
- 交换模块托架 2：IBM eServer BladeCenter 的 Nortel Networks Layer 2/3 Copper GbE Switch Module

在 86773XU BladeCenter 中设置 10 个 LS20 刀片的成本约为 6 万美元。推荐使用 1Gbps 网卡。确定所需网卡的准确方法是测量游戏的内部服务器流量。如果流量达到网卡容量的 25%，我们建议使用更快的网卡（即，如果流量超过 25Mbps，请使用 1Gbps 网卡）。请注意，大多数 100Mbps 网卡无法处理超过 50Mbps 的持续吞吐量。

磁盘存储

电源

内存

对于一般机器（CellApp、BaseApp 和各种管理器），不推荐使用 RAID 磁盘设置。这些机器都不广泛使用磁盘子系统（并且肯定不受磁盘限制）。使用标准驱动器，其大小足以存储整个世界数据（通常在 1 到 10G 的数量级）。如果驱动器出现故障，可以更换并从主副本复制数据。具有数据主副本的机器应使用 RAID 5 系统以获得速度 and 数据完整性。热插拔驱动器将在驱动器出现故障时便于更换。数据库服务器还需要使用 RAID 5 以确保数据完整性，并使用逻辑卷管理进行快照。我们还建议使用 10k 或 15k RPM 驱动器（SATA 或 SCSI）。游戏数据库存储所有实体的备份副本。此数据库可能很大，可能是 10G 到 1TB，但应在开发期间测量。这可以通过将实体数量乘以其大小来估算。我们建议为数据库机器和主数据服务器

使用双冗余 PSU。所有其他机器可以使用标准单 PSU，因为这些机器的故障并不关键。让 BigWorld 容错系统在软件方面完成工作更便宜。对于 CellApp 的内存需求，我们推荐以下计算：大约 32MB 到 128MB，用于 Linux 舒适运行（取决于您对内核和系统服务的精简程度）。大约 32MB 用于 CellApp 或 BaseApp 舒适运行，没有实体和没有加载的空间。足够的 RAM 用于您的实体和世界几何体（请记住，cell 需要加载足够的几何体以覆盖它支持的所有实体的 AoI）。此数量取决于您的网格密度以及每个实体存储的数据量。对于普通游戏，2GB 的 RAM 并不少见。BaseApp 通常需要大约 512M，具体取决于存储的实体数据量。所有其他机器需要大约 512M。

NOC 带宽

VMWare

这很容易计算。将玩家数量乘以每个玩家的所需带宽。传出带宽通常高于传入带宽。可以将 VMWare 用于单个开发人员测试目的，但是不推荐将 VMWare 用作可扩展的生产配置，因为可能引入计时延迟。还需要注意的是，如果您打算使用 VMWare 映像，您创建的包的架构应与您将在其上运行映像的预期机器相同。虽然这看起来有悖常理，但这很重要，因为跨架构模拟会显著减慢服务器速度，并且通常由于缺乏响应性而导致进程死亡。

附录 B. 安装 CentOS

安装

安装后设置



即使是有经验的用户也应浏览以下各节以确保安装了所需的软件包。您可能希望参阅 CentOS 文档（CentOS 5: <http://www.centos.org/docs/5/>，CentOS 7: <http://wiki.centos.org>）以获取有关安装和配置 CentOS 的其他说明和指南。使用安装 DVD 或其他介质（例如 PXE 引导）启动计算机。有关更多详细信息，请参阅 CentOS 文档。从 DVD 安装时，您可能需要在 BIOS 中选择 CD/DVD ROM 驱动器作为可引导设备。本安装指南基于图形安装程序。在第一个引导屏幕上按 ENTER 以选择 **Install in graphical mode**（以图形模式安装）。如果您遇到显卡驱动程序问题，可以重新启动并尝试仅文本安装程序。如果这是第一次使用 CD/DVD，则使用内置测试选项是值得的。测试大约需要 15 分钟。如果不想测试，只需选择 Skip。

Language and keyboard type.（语言和键盘类型。）

所选语言将用于安装过程以及安装系统的默认语言。

Installation method（安装方法）

如果您使用 DVD 进行安装，请选择 **Local CDRom**，或者您可以选择本地的 CentOS 镜像。

Disk partitioning (磁盘分区)

您可以根据需要分区磁盘。Remove all partitions on selected drives and create default layout 选项应该适用于大多数情况。您可以通过选中 Review and modify partitioning layout 复选框来修改默认布局。



如果机器将托管运行 MySQL 的数据库服务器并且您使用辅助数据库，则需要使用 LVM 分区，并且还需要为 LVM 快照分配一些可用空间。在查看分区布局时，您可以在其中一个逻辑驱动器上添加未分配的空间。有关快照工具的更多详细信息，请参阅 Database Snapshot Tool。

Boot loader configuration (引导加载程序配置)

为您的机器选择适当的选项（默认情况下，引导加载程序将安装到 MBR）。

Network configuration (网络配置)

确保至少列出了一个网络设备，并且为其启用了 IPv4。在生产中，对于 BaseApp 和 LoginApp 机器，应该有两个网络接口，一个用于外部流量，另一个用于内部服务器流量。在开发中，它们可以是同一个。主机名可以手动指定，也可以从 DHCP 设置。如果不使用 DHCP，则需要输入默认网关和 DNS 地址。

Time zone selection (时区选择)

选择您的时区。我们建议您按照默认将系统时钟保留为 UTC。

Setting the root password (设置 root 密码)

您将需要访问 root 账户以安装某些 BigWorld 服务器组件，请确保记住此密码。

Package selection (软件包选择)

如果这是生产机器，我们建议您取消选中 Desktop - Gnome。您可以取消选中其他选项，BigWorld 服务器所需的特定软件包将在本指南的后面安装。在开发中，您可能希望将机器用作桌面开发机器，在这种情况下，您可以选择安装开发所需的任何软件包，例如 Desktop - Gnome 软件包组。

First boot configuration (首次引导配置)

安装程序将格式化磁盘分区、安装基本系统和系统软件包。此过程完成后，您将被要求重新启动机器。首次引导时，系统将提示您进行进一步配置。

Authentication (身份验证)

此工具设置您的操作系统将如何查找用户账户信息。BigWorld 组件假设用户名到 UID 的映射在网络中是唯一的，即两台不同机器上同名的两个用户将具有相同的 UID，反之亦然。如果在多台机器上创建同名账户，请确保它们都具有相同的 UID（通过手动指定其 UID）。此外，BigWorld 服务器还假设由同一用户（通过其 UID 标识）启动的服务器组件属于同一服务器实例，即使这些组件在不同的机器上运行。要运行多个 BigWorld 服务器实例，需要多个用户账户。您可以设置远程账户信息服务器，如 LDAP。我们建议在开发期间使用 LDAP，以确保集群中的每台机器都具有相同的用户集。通常，每个开发者用户都有一个账户，他们可以在其中独立于其他用户运行自己的服务器。有关如何配置 LDAP 服务以对用户进行身份验证的更多信息，请参阅 OpenLDAP 文档。

Firewall configuration (防火墙配置)

对于开发机器，应禁用防火墙。默认防火墙阻止所有 UDP 流量，这会阻止 BigWorld 服务器运行。您可以通过将 **Security Level** 选项设置为 **Disabled** 来禁用防火墙。对于生产机器，您需要为您的特定安全要求设置专门的防火墙规则。BigWorld 服务器特定防火墙设置的指南在本文档的 Cluster Configuration 的 Security 节中给出。已知 BigWorld 服务器可以使用默认的 SELinux 设置 (enforcing) 工作。

安装后设置

安装更新

配置服务

安装构建工具

System services (系统服务)

对于生产机器，为了避免系统后台服务引起的意外负载峰值，我们建议您禁用任何非必要的服务。您可以配置在引导时启动哪些服务。建议禁用的服务包括：cups、bluetooth、yum-updatesd

Finishing the installation (完成安装)

退出首次引导配置屏幕后，您将看到登录提示。以 **root** 用户身份登录以继续安装。虽然不是严格要求，但安装最新更新是个好主意。您可以通过以 **root** 身份运行以下命令来更新软件包：

```
# yum update
```

为了避免系统后台服务引起的意外负载峰值，应禁用非必要的服务。可以通过以 **root** 身份运行以下命令来修改服务配置：

```
# firstboot --reconfig
```

这将调出与安装操作系统后出现的相同配置菜单。选择 **System services** 选项，然后取消选中您不希望在引导时启动的服务。请参阅上面的 **Installing** 中建议禁用的服务列表。

更改 UID

要构建 BigWorld 服务器，必须安装 GCC 和 Make。这些应该是您的 Linux 安装上的默认编译器和 make 实用程序。

```
# yum install gcc-c++ make
```

BWMachined

的一个要求是，集群中的所有机器必须具有相同的用户账户信息，特别是特定用户名的数字用户 ID (UID) 在每台机器上都相同。设置集群时，集群中的一个系统最终可能会得到与其他系统不同的 UID 到用户名的映射。如果您不使用 LDAP 或类似工具同步登录名，则尤其可能发生这种情况。如果您在这些机器上使用 GNOME 桌面环境，则在更改用户名的 UID 时可能会出现问題。本节概述了更改用户名的 UID

并避免这些问题的步骤。如果所有机器的设置使得每个用户账户在每台机器上都具有相同的 UID，则可以跳过本节。

- 1 确保没有用户以图形方式登录。
- 2 如果您处于图形模式，请按 CTRL + ALT + F1 切换到文本控制台。
- 3 以 **root** 身份登录。
- 4 为您的用户选择新的用户 ID 和组 ID，确保新用户 ID 没有被任何其他用户使用，类似地，组 ID 没有被另一个组使用。您可以通过查看 `/etc/passwd` 和 `/etc/group` 来检查。按照惯例，用户的主要组 ID 和用户 ID 相同，尽管不必如此。
- 5 通过调用以下命令更改用户的用户 ID 和组 ID 以及用户的主要组：

```
# groupmod -g <new GID> <groupname>
# usermod -u <new UID> -g <new GID> <username>
```

- 1 确认新用户具有新的 UID 和 GID：

```
# id <username>
```

- 1 发出以下命令以删除任何无效的用户状态：

```
# rm -rf /tmp/*<username>*
```



rm 是删除文件的命令，**-rf** 标志指示递归搜索，并强制删除所有匹配名称的目录和文件，而不会提示您。因此，这些标志应仅在极度谨慎的情况下使用。星号表示在搜索要删除的文件时的通配符字符集。例如，对于用户 `Alice`，此命令将删除名为 `/tmp/mapping-Alice` 的文件（如果它存在）。

- 1 您现在需要将主目录的所有权更改为新用户和组：

```
# chown -R <username>:<groupname> /home/<username>
```

- 1 如果需要，按 CTRL + ALT + F7 返回图形登录。

附录 C. 将文件复制到 Linux

USB 闪存盘 / 外置硬盘

在 Windows 上托管的 Python SimpleHTTPServer、Windows 网络共享 对于不熟悉使用 Linux 的人来说，即使是将文件从 Windows 机器复制到 Linux 机器这样的简单任务也可能令人畏惧。本节旨在通过提供多种（希望是）方便的替代方法来协助此过程，这些方法可用于帮助在机器之间传输文件。有多种方法可用于从 Windows 复制文件到 Linux。这些方法包括但不限于：

- 在 Windows 上托管的 Python SimpleHTTPServer

- Windows 网络共享

下面简要概述了每种方法。我们鼓励您独立研究每种方法，以便能够在需要时随时执行这些操作而无需 BigWorld 的帮助。以下示例将假设您从位于包顶层的 `rpm` 目录中的标准 BigWorld 包复制 RPM 文件。将文件复制到 Linux 机器上的位置无关紧要。但是，Simple Installation 节中列出的命令假设您将文件复制到 `/root`。



在 Linux 中，整个文件系统组织在单个层次结构中。`/` 位置指的是根级别，子目录在此之后列出。例如，要切换到 `/usr/include` 目录，您可以使用 `cd`（即“change directory”，更改目录）命令，方式如下：

```
$ cd /usr/include
```

安装 NTFS 支持

在 Windows 上托管的 Python SimpleHTTPServer

使用此方法时，您可能需要注意所使用驱动器的文件系统类型。FAT32 文件系统被 Linux 原生支持，而 NTFS 则不支持。通常，在 Windows 下格式化的驱动器将具有 NTFS 文件系统而不是 FAT32。如果您的驱动器出现这种情况，请参阅安装 NTFS 支持。这种方法可能是用于在机器之间一次性复制少量文件的最简单方法。一旦您将文件从 Windows 机器复制到 USB 设备并将其插入 Linux 机器，该设备应该会自动挂载。如果您登录到图形账户，您应该会看到该设备作为新图标出现在桌面上。仅命令行用户可能需要执行更多步骤来发现设备挂载的位置。`mount`

命令应该为您提供所有设备到目录的映射列表，以使您能够发现文件所在的目录。要为您的 CentOS 安装 NTFS 支持，首先确保您已按照安装 EPEL 仓库中所述安装了 EPEL 仓库，然后以 root 身份运行以下命令：

```
# yum install ntfs-3g ntfsprogs
```

在 Windows 上托管的 Python SimpleHTTPServer

此方法假设您已在 Windows 机器上安装了 Python，并在 `$PATH` 环境变量中拥有 Python 可执行文件。安装或下载 BigWorld Technology 包后，通过单击“开始”菜单中的“运行”打开命令行窗口，并在提示符中输入 `cmd`。导航到新安装中的 `rpm` 目录并输入：

```
C:\BigWorld\rpm> python.exe -m SimpleHTTPServer
Serving HTTP on 0.0.0.0 port 8000 ...
```

Windows 网络共享

您现在可以在 Linux 机器上使用 `wget` 程序或 Web 浏览器通过 HTTP 复制文件。如果不确定 Windows 机器 IP 地址，可以在命令行上使用 `ipconfig` 程序。例如，要复制 2.1 bwmachined RPM 文件，您将使用以下命令，将 10.40.3.145 替换为您自己的 IP 地址：

```
$ wget http://10.40.3.145:8000/bigworld-bwmachined-2.1.0.x86_64.rpm
| --2011-11-05 11:16:28-- | http://10.40.3.145:8000/ |
bigworld-bwmachined-2.1.0.x86_64.rpm
Connecting to 10.40.3.145:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1096149 (1.0M) [application/octet-stream]
Saving to: `bigworld-bwmachined-2.1.0.x86_64.rpm'
| 100%[=====>] 1,096,149 | 4.93M/s |
in 0.3s
2011-11-05 11:16:29 (11.8 MB/s) - `bigworld-bwmachined-2.1.0.x86_64.rpm' saved [1096149/1096149]
```

从 Windows 网络共享复制文件可能非常方便，但根据您的网络设置，最初设置可能会有问题。要使用此文件复制机制，请确保使用高级网络文件共享而不是默认的简单文件共享来共享 Windows 文件。有关在 Windows 上配置网络共享的更详细说明，请参阅 *Server Programming Guide* 的 *Shared Development Environments* 章节。为简单起见，要在 Linux 机器上检索文件，您应该使用图形登录并使用“Places>Network Servers”菜单选项导航到您的 Windows 机器。

附录 D. 创建自定义 BigWorld 服务器安装

自定义 RPM

手动安装

附录 D. 创建自定义 BigWorld 服务器安装

服务器工具 随着游戏开发过程的推进，可能需要执行 BigWorld 服务器的自定义安装。这可能是由于 BigWorld 服务器二进制文件已重新生成，或者生产环境机器需要从 RPM 文件中提供的默认值更改安装位置。本章概述了一些可用于自定义每个 BigWorld 服务器组件安装的方法。无论您选择使用哪种方法，我们都假设您已正确获取了 BigWorld Technology 包。这是为了确保没有行尾问题，例如在尝试使用 Linux 中的脚本时使用 Windows CRLF 行尾，而 Linux 假设仅 LF 行尾。本章还将假设您对 Linux（特别是 CentOS 文件系统层次结构）有扎实的理解，并且有信心在命令行环境中根据需要定位文件。

BigWorld 分发用于构建正式发布的 RPM 的 RPM 文件和 Python 脚本。这使您能够根据需要为您的环境自定义 RPM。如果您希望使用此自定义方法，请参阅 *Server Operations Guide* 的 RPM 章节，该章节完整概述了 BigWorld RPM 构建脚本的工作方式以及可用于在大型网络环境中自动分发 RPM 的方法。

BWMachined

服务器

在所有 BigWorld 服务器组件中，BWMachined 可能由于其简单性而最容易自定义安装。要以 root 身份手动安装 BWMachined，您可以在 `game/tools/bigworld/server/install` 目录中运行

`bwmachined2.sh` 脚本，如下所示：

```
# ./bwmachined2.sh install
```

此操作将在 `CentOS 5` 上执行以下步骤：停止任何已安装的 `BWMachined` 守护程序，如果它存在则卸载它。但是，这不会卸载 `BWMachined` 的 RPM 包。

- 在 `/etc/rc.d/init.d/` 中创建 `BWMachined` `init` 脚本。
- 在 `/etc/rc[1-5].d/` 中创建符号链接。它设置为在 `rc1.d` 中停止，在 `rc[2-5].d/` 中启动。如果需要，可以手动更改这些。
- 将 `BWMachined` 可执行文件复制到 `/usr/sbin/` 目录。以守护程序模式启动可执行文件，就像使用 `start` 参数调用 `init` 脚本一样。

在 `CentOS 7` 上，它将执行以下步骤：停止任何已安装的 `BWMachined` 守护程序，并卸载它。但是，这不会卸载 `BWMachined` 的 RPM 包。

- 在 `systemd unit` 目录（通常是 `/usr/lib/systemd/system`）中创建 `BWMachined` `systemd` 脚本。
- 重新加载 `systemd` 管理器配置以检测新服务。
- 启用 `bwmachined` 服务，使其在引导时自动启动。
- 立即启动 `bwmachined` 服务。

`BigWorld` 服务器的自定义安装不受任何安装脚本或文件系统位置的约束。所必需的只是，将在集群中每台机器上运行 `BigWorld` 服务器的用户可以访问服务器二进制文件和游戏资源。这意味着可以在用户的主目录或主机上的任何其他位置安装 `BigWorld` 服务器二进制文件。

服务器工具

需求与注意事项

需求

依赖项

安装过程

由于通常强制 `BigWorld` 网络集群中单个用户在机器之间的 `.bwmachined.conf` 文件相同，因此最好确保每台机器上 `BigWorld` 服务器二进制文件的安装位置一致。这也确保了机器的持续维护更加容易。如果您选择管理自己的 `BigWorld` 服务器二进制文件安装，您可能希望参阅 `Distributing Game Resources` 文档的 `Release Planning` 部分中提到的一些选项。由于安装服务器工具所涉及的子系统和配置的复杂性，服务器工具可能最受益于从 `RPM` 安装。以下说明描述了依赖项列表以及基本安装指南，尽管我们不再支持服务器工具的手动安装。

- 在服务器工具机器上安装并运行 `BWMachined`
- 服务器工具的专用用户账户
- MySQL 数据库

- Python 2.4 或更高版本 (RedHat / CentOS 默认)
- TurboGears v1.x (RedHat / CentOS 默认)
- python-ldap v2.2 或更高版本 (RedHat / CentOS 默认)

WebConsole 默认使用 SQLite 数据库管理所有持久数据，如用户信息、首选项等。统计信息收集进程 StatLogger 依赖 MySQL 数据库服务器来存储进程和机器统计信息。

安装过程不会像 RPM 安装过程那样以逐步方式概述。如果您尝试自定义安装 BigWorld 服务器工具，则假设您具有足够的系统理解和 BigWorld 经验来执行下面描述的每个步骤。自定义安装 BigWorld 服务器工具需要您执行以下步骤：按照安装 EPEL 仓库中所述安装 EPEL 仓库。

- 安装随 BigWorld 包分发的 **MySQL-python** RPM 包，以防止任何 WebConsole 和 StatLogger 连接问题。
- 安装所有依赖项。即使执行自定义安装，我们也建议您通过用于 BigWorld RPM 的默认 CentOS 仓库安装依赖链，因为这将确保安装的软件包版本正确。应安装以下 yum 软件包：python-setuptools、python-sqlobject、TurboGears、python-ldap
- yum 更新您的发行版以确保已应用所有安全和错误修复。
- 在您的域中创建一个将用于运行工具的用户账户。例如，创建能够在服务器集群中的所有机器上解析并且在所有集群机器上具有唯一用户 ID 的用户 **bwtools**。
- 确保 MySQL 服务器正在运行并为所有适当的运行级别启用。
- 为 BigWorld 服务器工具创建 MySQL 服务器账户。
- 将 BigWorld 服务器工具安装到 BigWorld 工具用户目录中。
- 为 **_bwlog.so** 设置 SELinux 安全上下文。
- 更新 WebConsole 配置文件。
- 更新 StatLogger 配置文件。
- 更新 MessageLogger 配置文件。
- 将服务脚本从 **game/tools/bigworld/server/install** 复制到 **/etc/init.d** 目录并设置服务运行级别。
- 启动 MessageLogger、StatLogger 和 WebConsole。
- 为服务器工具日志文件配置日志轮换。
- 验证服务器重启后所有服务都在运行。

附录 E. 理解 BigWorld Machine Daemon (BWMachineD)

BWMachined 的工作原理

配置 BWMachineD

附录 E. 理解 BigWorld Machine Daemon (BWMachineD)

有关 BWMachineD 角色的信息，请参阅 Server Overview 的 BWMachineD 章节。为了使 BWMachineD 进程能够作为网络集群中用户的启动和定位服务器进程的代理，BWMachineD 需要一种机制来能够在每台运行它的机器上按用户查找游戏资源。为 BWMachineD 选择的方法是在将要启动任何 BigWorld 进程的每个用户的主目录中有一个配置文件。此配置文件指示 BWMachineD 在哪里可以找到 BigWorld 服务器二进制文件以及每台机器上要使用的游戏资源。虽然不常见，但这种方法允许每台机器可能具有不同的服务器二进制文件和游戏资源安装位置。当与 BigWorld 服务器工具交互并请求启动服务器时，将向 BWMachineD 发送消息，BWMachineD 反过来查询与请求用户关联的配置文件。然后，当 BWMachineD 尝试启动服务器进程时，将使用配置文件中指定的路径。有关 BWMachineD 配置文件布局的更多详细信息，请参阅以下章节。BWMachineD 在集群环境的持续运行中起着至关重要的作用，因此了解如何配置它以及哪些配置选项与您的服务器环境相关非常重要。有两个与 BWMachineD 操作相关的配置文件：

创建 ~/.bwmachined.conf

~/.bwmachined.conf 此文件用于指定与在 BigWorld 集群中工作的个人用户应如何在任何可用集群机器上找到运行所需的服务器资源相关的选项。

/etc/bwmachined.conf 此文件主要用于指定与运行 BWMachineD 的机器应如何在集群环境中操作相关的设置。启动服务器和相关组件时，需要两条重要信息：在哪里找到 BigWorld 服务器可执行文件。哪些目录包含要与服务器一起使用的游戏资源。

指定这些设置的首选方法是在 ~/.bwmachined.conf 文件中。



对于不熟悉 Linux 的用户：

- ~（波浪号）表示用户的主目录，例如名为 **johns** 的用户通常具有位于 **/home/johns** 的主目录
- 文件名前的句点字符表示它是隐藏文件，这使它不会被许多目录列表应用程序（包括 **ls**）显示（除非指定了 **-a** 选项）。

以下是用户 **johns** 的 **~/.bwmachined.conf** 文件示例。此文件需要在创建用户账户时手动创建。请注意使用分号和冒号的不同位置：指定这些设置的首选方法是在 ~/.bwmachined.conf 文件中。



对于不熟悉 Linux 的用户：

- ~（波浪号）表示用户的主目录，例如名为 **alice** 的用户通常具有位于 **/home/alice** 的主目录

- 文件名前的句点字符表示它是隐藏文件，这使它不会被许多目录列表应用程序（包括 `ls`）显示（除非指定了 `-a` 选项）。

创建 `/etc/bwmachined.conf`



以下是用户 `alice` 的 `~/.bwmachined.conf` 文件示例。此文件需要在创建用户账户时手动创建。请注意使用分号和冒号的不同位置：

```
# .bwmachined.conf
# Format: BW_ROOT;BW_RES_PATH:[BW_RES_PATH] ...
/opt/bigworld/current/server;/home/alice/fantasydemo/res:/opt/bigworld/current/server/res
```

分号前的路径应指向已安装 `bigworld` 文件的根目录。通常，您将在此根目录下有 `bigworld` 目录。分号后的路径（由冒号字符分隔）指定将用于查找游戏使用的资源的资源路径。使用标准服务器工具（如 `control_cluster.py` 或 `WebConsole`）启动 `BigWorld` 服务器时，`BWMachined` 负责启动服务器二进制文件，并使用位于 `~/.bwmachined.conf` 中的信息以及主机系统的架构来确定如何为请求用户启动服务器。



需要在集群环境中运行服务器的每个用户都需要为其创建和配置 `~/.bwmachined.conf` 文件。

`/etc/bwmachined.conf` 全局配置文件用于设置定义 `BWMachined` 将如何在其运行的主机上操作的选项。例如，如果您的集群中有多台机器，并且在开发期间您希望将某些机器隔离为开发者使用的组，则这将在全局配置文件中应用。以下列表提供了可在 `/etc/bwmachined.conf` 文件中应用的主机配置选项的快速摘要：用户定义类别、`Reviver` 配置、`BigWorld` 服务器计时方法、多接口主机的接口配置

Reviver 配置

计时方法

机器分组



配置文件仅在 `BWMachined` 启动时读取。如果要让它确认您的更改，则必须重新启动它。当 `Reviver` 进程启动时，它查询本地 `BWMachined` 进程，并且仅支持在名为 `[Components]` 的特殊用户定义类别中具有条目的组件。指定 `Reviver` 应支持所有服务器组件的示例配置将如下定义：

```
[Components]
baseApp
baseAppMgr
cellApp
cellAppMgr
dbApp
dbAppMgr
loginApp
```



BaseApp 和 CellApp 不会被 Reviver 重启，[Components] 条目由 WebConsole 和 control_cluster.py 用于确定应由 BWMachined 在该主机上启动哪些进程。但是，此列表仅是服务器工具的提示，如果需要，仍可能在该主机上启动这些进程。

如果 [Components] 类别不包含任何条目，则 Reviver 将支持所有服务器组件。默认情况下，时间服务由 clock_gettime 系统调用提供。默认值是推荐的计时方法，适用于所有支持的平台，有关更多选项，请参阅 Server Operations Guide 的 Clock 章节。

多接口主机的内部接口配置

运行 BWMachined 的机器的成员资格也可以选择在 /etc/bwmachined.conf 中指定。有关机器组用途以及如何指定它们的更多详细信息，请参阅 Server Operations Guide 的 Machine Groups and Categories 章节。

多接口主机的内部接口配置

BigWorld 机器守护程序用于内部机器和外部机器（如运行 BaseApp 和 LoginApp 的机器）。BigWorld 组件用于发现服务器进程和进程启动注册的协议涉及向机器守护程序发送 UDP 广播。机器守护程序必须确定从哪个接口接收这些广播。默认情况下，机器守护程序将通过在每个接口上发送广播数据包并等待此广播数据包返回来确定哪个接口是内部接口。接收到第一个广播数据包的接口被假定为内部网络。对于具有多个接口的外部机器，这可能导致选择错误的接口。在这些情况下，最好检查您的广播路由规则（请参阅路由）并考虑添加防火墙规则以阻止从其他接口上的 BWMachined 端口（20018 和 20019）接收。

例如，如果 eth1 不是您的内部接口：

```
# /sbin/iptables -I INPUT 1 -p udp -i eth1 -m multiport --destination-ports 20018:20019 -j DROP
```



在大多数情况下，最好阻止所有端口，然后仅打开所需的端口。请参阅安全。

在极少数情况下，可以使用连接到内部网络的接口的点分十进制地址或名称设置

[InternalInterface] 配置选项。例如，使用接口名称：

```
[InternalInterface]
eth0
```

使用点分十进制表示法：

服务配置

延迟数据包以避免网络泛洪

```
[InternalInterface]
192.168.0.1
```

如果未指定此选项，将执行内部接口的自动发现。如果指定了此选项，但未找到与

[InternalInterface] 中设置的值匹配的接口，则将向 syslog 记录错误，并且机器守护程序进程将终止。

此选项现在也可以在 `/etc/bwmachined.conf` 的 `[bwmachined]` 节中指定，如下所示：

```
[bwmachined]
internal_interface = <value>
```

当 `ServiceApp` 启动时，它查询本地 `BWMachined` 进程，并将启动在名为 `[Services]` 的特殊用户定义类别中具有条目的服务。指定仅应在新的 `ServiceApp` 上启动 `ExampleService` 和 `NoteStore` 服务的示例配置定义如下：

```
[Services]
ExampleService
NoteStore
```

有关 `ServiceApp` 和服务的更多信息，请参阅 `Server Overview` 的 `ServiceApp` 章节。



如果没有 `[Services]` 类别，则 `ServiceApp` 将启动所有服务。`[MaxPacketDelay]` 选项允许 `BWMachined` 随机延迟对网络广播消息的回复，最多延迟指定的毫秒数。这对于在短时间内产生大量数据包的大型网络集群很有用。随机延迟对这些类型数据包的回复可避免网络硬件被对单个主机的回复淹没。

附录 F. 使用 httpd 设置反向 HTTPS 代理服务器

生成 SSL 证书和私钥

附录 F. 使用 httpd 设置反向 HTTPS 代理服务器

安装 `httpd` 和所需模块、配置 `httpd` 并将 `WebConsole` 与其集成 本节描述如何在 Linux 上使用 `httpd` (Apache HTTP Server) 设置反向 HTTPS 代理服务器。它还描述了如何将 `WebConsole` 与 `httpd` 服务器集成。这些说明假设您的 Linux 服务器上已安装并配置了 `Yum`，并且 `Apache HTTP Server` 和 `openssl` 软件包在您的 `Yum` 仓库中可用。设置反向 HTTPS 代理服务器并将 `WebConsole` 与其集成有三个步骤：

- 1 生成 SSL 证书和私钥。
- 2 安装 `httpd` 和所需模块。
- 3 配置 `httpd` 并将 `WebConsole` 与其集成。

下面详细描述每个步骤。您需要 SSL 证书和私钥以使 `httpd` 能够使用 HTTPS。您可以从证书颁发机构 (CA) 购买 SSL 证书，也可以生成自签名证书。此处提供的说明描述了如何使用 `openssl` 生成自签名证书。

Install openssl (安装 openssl)

如果您的 Linux 服务器上尚未安装 `openssl`，请以 `root` 身份运行以下命令进行安装：

```
# yum install openssl
```

Generate an SSL private key (生成 SSL 私钥)

您可以通过运行以下命令生成 SSL 私钥：

安装 httpd 和所需模块

```
$ openssl genrsa -out server.key 1024
```

密钥文件 `server.key` 在您当前的工作文件夹下生成。

Generate a Certificate Signing Request（生成证书签名请求）

现在使用您的密钥创建证书签名请求。您可以将密码选项留空以创建不需要密码的证书：

```
$ openssl req -new -key server.key -out server.csr
```

您可以在此步骤中输入或忽略屏幕上显示的选项。

Generate an SSL certificate（生成 SSL 证书）

现在通过运行以下命令使用上面生成的密钥和 `csr` 文件创建 SSL 证书：

```
$ openssl x509 -req -days 366 -in server.csr -signkey server.key -out server.crt
```

现在您有了自签名 SSL 证书（`server.crt`）和密钥（`server.key`），`httpd` 可以使用它们。要以 `root` 身份安装 `httpd` 和 `mod_ssl`，请运行以下命令。

```
# yum install httpd
# yum install mod_ssl
```

您需要停用 `mod_ssl` 的配置文件（`/etc/httpd/conf.d/ssl.conf`），因为它与我们下面将进行的配置冲突。您可以通过重命名来停用它。为此，请以 `root` 身份运行以下命令：

```
# mv /etc/httpd/conf.d/ssl.conf{,.bak}
```

配置 httpd 并将 WebConsole 与其集成

配置 httpd 并将 WebConsole 与其集成

要使 `httpd` 提供反向 HTTPS 代理服务，您需要修改其配置文件（`httpd.conf`），如下所述。这些更改必须以 `root` 身份进行。`httpd.conf` 位于 `/etc/httpd/conf/` 下。

Load required modules（加载所需模块）

`httpd` 依赖 `SSL` 模块和 `Proxy` 模块来提供反向 HTTPS 代理服务。这些模块必须在 `httpd` 启动时加载到 `httpd` 中。在 `httpd.conf` 中默认启用加载代理相关模块。要使 `httpd` 在启动时加载 `SSL` 模块，请将以下行添加到 `httpd.conf`：

```
LoadModule ssl_module modules/mod_ssl.so
```

Listen on a specified port for HTTPS（在指定端口上侦听 HTTPS）

在 `httpd.conf` 中将默认侦听端口从 `80` 更改为另一个值（例如，您可以使用 `443`，即 HTTPS 服务的默认端口）。

```
Listen 443
```

Make httpd provide the HTTPS service（使 `httpd` 提供 HTTPS 服务）

要使 `httpd` 提供 HTTPS 服务，请将 `VirtualHost` 添加到 `httpd.conf`，如下所示：

```

<VirtualHost *:443>
    ServerName https://<hostname or IP of this Linux server>:443
    SSLEngine on
    SSLCertificateFile /home/alice/keys/server.crt
    SSLCertificateKeyFile /home/alice/keys/server.key
</VirtualHost>

```

在此示例中，SSL 证书和密钥保存在 `/home/alice/keys/` 下。您可以为服务器证书和密钥选择不同的文件夹，前提是此处的路径指向它们。

Make httpd act as reverse HTTPS proxy server of WebConsole（使 httpd 充当 WebConsole 的反向 HTTPS 代理服务器）

通过将以下行附加到 `httpd.conf` 来配置 HTTPS 服务以作为 WebConsole 的反向代理工作：

```

<IfModule mod_proxy.c>
    # Disable forward proxy requests
    ProxyRequests Off
    # Allow requests from all hosts
    <Proxy *>
        Order Allow,Deny
        Allow from all
    </Proxy>
    # Configure reverse proxy requests for all requests
    ProxyPass / <Put full WebConsole URL here, like http://10.1.2.3:8080>
    ProxyPassReverse / <Put full WebConsole URL here, like http://10.1.2.3:8080>
    # Require SSL between browsers and the proxy server for all requests
    <Location ~ "^/">
        SSLRequireSSL
    </Location>
</IfModule>

```

现在您可以通过以 `root` 身份运行以下命令来启动反向 HTTPS 代理服务器：在 CentOS 5 上：

```
# /etc/init.d/httpd start
```

在 CentOS 7 上：

```
# systemctl start httpd
```

您可以通过浏览器使用您刚刚在 `VirtualHost` 中指定的“`ServerName`”访问此代理服务器。如果您上面指定的 `WebConsole` 也在运行，您应该能够通过此代理服务器 URL 看到 WebConsole 网页。

附录 G. 安装和配置 MongoDB

安装 MongoDB

配置 MongoDB

MongoDB 的推荐 Linux 系统配置

附录 G. 安装和配置 MongoDB

MongoDB 分片设置、启用认证、使用 MongoDB 备份和恢复日志 MongoDB 可以用作 MessageLogger 的后端存储。有关配置 MessageLogger 的信息，请参阅配置 MessageLogger。本章描述如何安装和配置 MongoDB。本附录提供 MongoDB 文档版本 2.4 的链接。如果您安装不同版本的 MongoDB，请使用匹配的 MongoDB 文档。MongoDB 服务器必须安装在 CentOS 7 上。如果您想使用 MongoDB 作为消息日志记录器后端存储，还需要在 CentOS 7 上安装 BigWorld 服务器工具 rpm。您可以在不同的 CentOS 7 主机上安装 MongoDB 服务器和 BigWorld 服务器工具。在安装 MongoDB 之前，请确保已安装 epel。可以通过 epel 在 CentOS 7 上安装 MongoDB。安装 epel 后，运行以下命令安装 MongoDB。

```
#yum install mongod
#yum install mongod-server
```

MongoDB 在 <http://docs.mongodb.org/v2.4/administration/production-notes/#recommended-configuration> 上提出了几项关于如何提高性能的建议。本节概述如何在 CentOS 7 上实施这些更改。

配置 MongoDB 时的一个重要考虑因素是使用哪种文件系统来存储数据。MongoDB 文档建议使用 Ext4 或 XFS 文件系统。请注意，不推荐 Ext3：我们的内部测试发现，在 Ext3 上预分配数据文件会导致严重的性能问题。例如，我们发现 MongoDB 分配 1GB 文件并用零填充它需要超过 20 秒。关闭磁盘访问时间。例如，如果 `/var/lib/mongod` 是包含数据库文件的存储卷：

- 1 将 "noatime" 添加到 `/etc/fstab` 中的驱动器选项：

```
| /dev/mapper/vg01-lv_mongod | /var/lib/mongod | ext4 | defaults,noatime 1 2 |
```

- 1 使用以下命令重新挂载驱动器：

```
mount -o remount /var/lib/mongod
```

通过更新 `/usr/lib/systemd/system/mongod.service` 的 `[Service]` 节以包含这些选项来增加文件和进程限制：

```
LimitNOFILE=64000
LimitNPROC=64000
```

禁用透明大页。为此：

- 1 通过将以下内容添加到 `/etc/rc.local` 的末尾来永久禁用大页：

```
if test -f /sys/kernel/mm/transparent_hugepage/enabled; then
    echo never > /sys/kernel/mm/transparent_hugepage/enabled
fi
if test -f /sys/kernel/mm/transparent_hugepage/defrag; then
    echo never > /sys/kernel/mm/transparent_hugepage/defrag
fi
```

- 1 此文件默认不可执行，因此您需要使用以下命令更改权限：

```
chmod +x /etc/rc.d/rc.local
```

1 使用以下命令在当前会话中禁用它们：

```
echo never > /sys/kernel/mm/transparent_hugepage/enabled
echo never > /sys/kernel/mm/transparent_hugepage/defrag
```

确保 NTP 正在运行。为此，运行命令：

```
ps aux | grep ntp
```

并确认输出类似于：

```
| ntp | 3338 | 0.0 | 0.0 | 29228 | 1720 ? | Ss | Sep17 | 0 | :00 /usr/sbin/ntpd -u
ntp:ntp -g |
```

如果不是，则使用以下命令安装 NTP：

```
yum -y install ntp
chkconfig --level 235 ntpd on
service ntpd start
```

确保 NUMA 已禁用。为此，运行命令：

```
grep -i numa /var/log/dmesg
```

并确认输出类似于：

```
| [ | 0.000000] No NUMA configuration found |
```

如果输出指示 NUMA 已启用，请按照 <http://docs.mongodb.org/manual/administration/product-ion-notes/#mongodb-and-numa-hardware> 上的说明禁用它。

MongoDB 配置选项

MongoDB 分片设置

可以在 MongoDB 配置文件中或通过 MongoDB 命令行选项指定 MongoDB 配置选项。以下是 mongod 的一些重要配置选项。有关 MongoDB 配置选项的更多信息，请参阅 <http://docs.mongodb.org/v2.4/reference/configuration-options/>

- **dbpath** — 指定 MongoDB 服务器将数据存储到的数据目录的路径。
- **port** — 指定 MongoDB 服务器将侦听的端口。
- **configsvr** — 使用此选项启动 mongod 将使 mongod 作为配置服务器工作。
- **auth** — 设置为 true 以启用身份验证，设置为 false 以禁用身份验证。
- **logpath** — 指定 MongoDB 应将其自己的日志存储在哪里。
- **keyFile** — 指定用于存储身份验证信息的密钥文件的路径。此选项用于分片集群或副本集的 mongos 和 mongod 实例之间的进程间身份验证。

本节说明如何在 CentOS 7 上设置分片 MongoDB 环境，以便与 Message Logger 一起使用。分片跨多台机器存储数据，以便在重负载条件下提高读写性能。更多详细信息请参阅

<http://docs.mongodb.org/manual/sharding/>。

生产分片集群需要三种不同的服务：一个或多个 **routers**（路由器），两个或多个 **shards**（分片）和恰好三个 **config servers**（配置服务器）。路由器与客户端应用程序接口，并将操作定向到适当的一个或多个分片。只需要一个，但额外的路由器可以帮助分担客户端请求负载并增加冗余。分片存储数据。它们实际上是标准 MongoDB 服务的相同物，因此它们的系统要求（磁盘、内存等）相同。配置服务器存储集群的元数据，详细说明哪些分片包含什么数据。路由器使用此元数据将操作定向到特定分片。

配置 MongoDB 集群

http://docs.mongodb.org/manual/_images/sharded-cluster-production-architecture.png 说明了这些服务之间的关系。此处描述的基本布局使用三台主机（mongo01、mongo02、mongo03），每台运行所有三种服务。将每个服务隔离到单台主机可能会产生更好的性能和可靠性。

Set up and distribute key file（设置和分发密钥文件）

- 1 首先在其中一台主机上使用以下命令创建密钥文件：

```
openssl rand -base64 741 > /etc/mongodb-keyfile
```

- 1 将密钥文件分发到集群中的所有其他主机，然后在每台主机上运行以下命令：

```
chmod 600 /etc/mongodb-keyfile
chown mongodb:mongodb /etc/mongodb-keyfile
```

Set up Config Servers（设置配置服务器）

对每台主机执行以下步骤：

- 1 通过运行以下命令创建配置服务器配置文件：

```
cp /usr/lib/systemd/system/mongod.service /usr/lib/systemd/system/mongod-config.service
cp /etc/sysconfig/mongod /etc/sysconfig/mongod-config
cp /etc/mongodb.conf /etc/mongodb-config.conf
mkdir /var/lib/mongodb-config
chown mongodb:mongodb /var/lib/mongodb-config
```

- 1 替换 `/usr/lib/systemd/system/mongod-config.service` 中以下行的现有值：

```
PIDFile=/var/run/mongodb/mongodb-config.pid
EnvironmentFile=/etc/sysconfig/mongod-config
```

- 1 将 `/etc/sysconfig/mongod-config` 的内容替换为：

```
OPTIONS="--quiet -f /etc/mongodb-config.conf"
```

- 1 将 `/etc/mongodb-config.conf` 顶部的 “Basic Defaults” 节替换为：

```
#bind_ip = 127.0.0.1
#port = 27017

fork = true
```

```
pidfilepath = /var/run/mongodb/mongodb-config.pid
logpath = /var/log/mongodb/mongodb-config.log
dbpath = /var/lib/mongodb-config
```

```
journal = true
configsvr = true
```

```
keyFile = /etc/mongodb-keyfile
```

- 1 使用以下命令启动并启用 mongod-config 服务：

```
systemctl start mongod-config
systemctl enable mongod-config
```

- 1 如果服务启动失败，请检查 `/var/log/mongodb/mongodb-config.log` 中的错误。您还可以使用以下命令：

```
systemctl status mongod-config.service
```

- 1 如果您对 `mongo-config.service` 进行了任何更改，在尝试再次运行它之前，最好使用以下命令重置配置：

```
systemctl daemon-reload
```

Router (mongos) (路由器 (mongos))

对每台主机执行以下步骤：

- 1 通过运行以下命令创建路由器配置文件：

```
cp /usr/lib/systemd/system/mongod.service /usr/lib/systemd/system/mongos.service
cp /etc/sysconfig/mongod /etc/sysconfig/mongos
cp /etc/mongodb.conf /etc/mongos.conf
```

- 1 替换 `/usr/lib/systemd/system/mongos.service` 中以下行的值：

```
PIDFile=/var/run/mongodb/mongos.pid
EnvironmentFile=/etc/sysconfig/mongos
ExecStart=/usr/bin/mongos $OPTIONS
```

- 1 替换 `/etc/sysconfig/mongos` 的内容：

```
OPTIONS="--quiet -f /etc/mongos.conf"
```

- 1 替换 `/etc/mongos.conf` 顶部的 “Basic Defaults” 节。确保 `configdb` 值与您配置的配置主机名匹配。

```
#bind_ip = 127.0.0.1
#port = 27017
```

```
fork = true
```

```
pidfilepath = /var/run/mongodb/mongos.pid
```

```
logpath = /var/log/mongodb/mongos.log
configdb = mongo01,mongo02,mongo03
keyFile = /etc/mongodb-keyfile
```

- 1 使用以下命令启动并启用 mongos 服务:

```
systemctl start mongos
systemctl enable mongos
```

- 1 如果服务启动失败, 请检查 `/var/log/mongodb/mongos.log` 中的错误。您还可以使用以下命令:

```
systemctl status mongos.service
```

- 1 如果您对 `mongos.service` 进行了任何更改, 在尝试再次运行它之前, 最好使用以下命令重置配置:

用户创建

```
systemctl daemon-reload
```

Shard (mongod) (分片 (mongod))

分片服务器是标准的 mongo 服务, 因此它需要最少的配置更改。对每台主机执行以下步骤:

- 1 替换 `/etc/mongod.conf` 顶部的 “Basic Defaults” 节。这些说明将其端口更改为非默认值, 以防止与在同一主机上运行的路由器冲突, 但如果服务位于不同的主机上, 则并非严格必要。

```
#bind_ip = 127.0.0.1
port = 27018

fork = true

pidfilepath = /var/run/mongodb/mongod.pid
logpath = /var/log/mongodb/mongod.log
dbpath = /var/lib/mongodb

journal = true

keyFile = /etc/mongodb-keyfile
```

- 1 使用以下命令启动并启用 mongod 服务:

```
systemctl start mongod
systemctl enable mongod
```

- 1 如果服务启动失败, 请检查 `/var/log/mongodb/mongod.log` 中的错误并使用:

```
systemctl status mongod.service
```

一旦所有服务都配置并正确运行, 您需要创建具有管理集群能力的用户。

- 1 使用以下命令添加 admin 用户:


```
mongo admin
mongos> db.addUser( { user: "admin", pwd: "admin", roles: [ "userAdminAnyDatabase", "clusterAdmin"
```

分片初始化

- 1 退出，然后以 admin 身份重新连接以添加 bwtools 用户：

```
mongo -u admin -p admin admin
> db.addUser( { user: "bwtools", pwd: "bwtools", roles: ["dbAdminAnyDatabase", "clusterAdmin", "read"
```

- 1 请注意，如果您已经创建了用户，则需要更新其角色：

```
> use admin
> db.system.users.update( { user: "admin" }, { $set: { roles:
| [ "userAdminAnyDatabase", |
| "readWriteAnyDatabase" ] } } )
> db.system.users.update( { user: "bwtools" }, { $set: { roles
| : [ "dbAdminAnyDatabase", |
| "clusterAdmin", |
| "readWriteAnyDatabase" ] } } )
```

下一步是初始化 Message Logger 使用的分片。以 admin 身份连接以添加每个已配置的分片主机。在这种情况下，在 mongo01:27108、mongo02:27108 和 mongo03:27108 上有三个可用分片：

```
mongo -u admin -p admin admin
> sh.addShard("mongo01:27108")
{ "shardAdded" : "shard0000", "ok" : 1 }
> sh.addShard("mongo02:27108")
{ "shardAdded" : "shard0001", "ok" : 1 }
> sh.addShard("mongo03:27108")
{ "shardAdded" : "shard0002", "ok" : 1 }
```

可以随时使用此方法添加更多分片。集群现在可以使用了！如果有必要，您也可以删除分片。请注意，删除的分片需要将其所有数据重新定位到其他主机，这可能很耗时并影响集群性能。

```
mongo -u admin -p admin admin
```

Message Logger 配置

启用认证

```
> use admin
> db.runCommand({removeshard: 'shard0002'})
> sh.status()
```

状态现在应显示分片正在“draining”（排空），并且可能持续很长时间。最终，其所有数据将重新定位到其他分片，并且它将不再显示在状态摘要中。Message Logger 的分片配置与常规 MongoDB 配置相同，只是主机现在应引用其中一个路由器。目前在配置文件中无法指定多个路由器。要启用与 MongoDB 服务器或服务器集群的连接的身份验证，您需要在 MongoDB

管理数据库中创建用户并以启用身份验证的方式启动 MongoDB 组件。按照以下步骤设置用户名 `bwtools` 和密码 `bwtools` 的基本身份验证：

- 1 通过 MongoDB shell 创建用户。为此，使用命令 `use admin` 切换到 `admin` 数据库，并使用以下命令添加用户：

```
db.addUser( { user: "admin", pwd: "admin", roles: [ "userAdminAnyDatabase" ] } )
db.addUser( { user: "bwtools", pwd: "bwtools", roles: [ "dbAdminAnyDatabase", "clusterAdmin", "readw
```

在指定 Message Logger 配置中的 MongoDB 用户和密码时，您将需要上面创建的用户名和密码（在此示例中为 `user: "bwtools", pwd: "bwtools"`）。

- 1 以启用身份验证的方式启动您的数据库服务器/集群。如果仅启动单个数据库服务器，可以通过在启动 `mongod` 时指定 `--auth` 选项来启用身份验证：

```
mongod --auth <other options>
```

使用 MongoDB 备份和恢复日志

MongoDB 中 Message Logger 的数据库架构

用户日志数据

如果要启动集群，需要生成密钥文件并使用 `{{-keyFile <key file path>}}` 启动每个 MongoDB 组件。有关更多信息，请参阅

<http://docs.mongodb.org/v2.4/tutorial/enable-authentication-in-sharded-cluster/>。

- 1 将 MongoDB shell 连接到启用身份验证的服务器/路由器：

```
mongo <other options> -u bwtools -p bwtools --authenticationDatabase admin
```

本节描述如何备份和恢复存储在 MongoDB 中的日志数据。要备份和恢复 MongoDB 中的 Message Logger 日志数据，了解 Message Logger 日志数据如何存储在 MongoDB 中很重要。Message Logger 存储两种不同类型的数据：存储用户日志的用户日志数据和被用户日志数据引用的公共数据。它们存储在不同的数据库中。公共数据存储在一个数据库中，每个用户的日志存储在该用户自己的数据库中。允许多个 Message Logger 实例使用相同的 MongoDB 实例，如果它们具有不同的 `loggerID`。每个 Message Logger 将拥有自己的一组数据库，包括自己的公共数据数据库和用户日志数据库。为了支持这些，每个 Message Logger 的 `loggerID` 将被编码到其数据库名称中。每个 Message Logger 将仅创建和管理自己的数据库。当用户使用 LogViewer 或 `mlcat.py` 查询日志时，查询将返回该用户由任何 Message Logger 写入的日志。每个用户的日志数据存储在他们自己的名为 `bw_ml_user_<username>#<loggerID>` 的数据库中。在每个这些数据库中，有三种类型的集合：`entries`、`uid` 和 `server_start_ups`。每个 `entries` 集合在其名称中有一个时间戳以标识其创建时间。集合名称采用 `entries_<timestamp>` 格式（例如，`entries_20141027050547`）。仅在 Message Logger 启动和日志轮换期间创建新的

entries 集合。

以下是 entries_<timestamp> 的完整架构：

名称	描述	类型	备注
_id	_id	ObjectID	未指定时由 MongoDB 默认创建，12 字节
ts	timestamp	Date	在 BSON 中，Date 是一个 64 位整数，表示自 Unix 纪元以来的毫秒数。在 MongoDB 驱动程序 API 中，应注意如何使用 Date 类型
cnt	counter	Integer	用于维护日志的插入顺序。此数字是从 0 开始每秒自动递增的值
ctg	category	Integer	这是 categories 集合中一条记录的 id，类似于外键
src	source	Integer	日志的来源，C++、Python。这是 sources 集合中一条记录的 id，类似于外键
svt	severity	Integer	日志的严重性级别，如 DEBUG、ERROR 等。这是 severities 集合中一条记录的 id，类似于外键
host	host	Integer	源主机 IP 地址的整数值（因此目前我们仅支持 IPv4），对应于 hosts 集合中的一个 ip 值
pid	pid	Integer	进程 ID
fmt	format	Integer	此日志的格式字符串，对应于 format_strings 集合中记录的字符串的 id

名称	描述	类型	备注
cpt	component	Integer	这是 components 集合中一条记录的 id，类似于外键
aid	app id	Integer	应用 ID（例如 CellApp ID）。如果进程不是 app，则不会存在

Common Data（公共数据）

在 MongoDB 中备份和恢复 Message Logger 日志数据

entries_<timestamp> 集合的完整 schema 如下（接上页）：

名称	描述	类型	备注
msg	log	String	此日志消息的消息内容
md	meta data	Object	元数据字段，一个 JSON 对象

除了 entries 集合之外，还有另外两种集合：

集合名称	描述
server_start_ups	服务器启动时间，可以是 MongoDB 的 capped collection。
uid	用户 ID，只会有一条记录。

公共数据存储在数据库 bw_ml_common#<loggerID> 中。公共数据是用户日志共享的数据，包括格式字符串、主机名、组件名、日志类别、来源、严重性级别以及当前的数据库 schema 版本。这些数据存储在单独的集合中，通常每条记录包含一个 ID 和一个值。ID 用于在用户日志中以节省磁盘空间，而值（通常是字符串）用于日志显示。在备份用户日志时，此公共数据应一并备份，否则用户日志数据将失效且无法被 Log Viewer 或 Message Logger 工具使用。

目前没有一种简单的方法可以在不影响 MongoDB 服务器性能的情况下备份日志数据。本节总结了 MongoDB 推荐的备份和恢复方法。请注意，这些方法不适合定期备份/归档大型集群或大型数据集，因为操作复杂且可能对性能产生不利影响，完成时间也可能较长。

MongoDB 推荐的方法

要归档大型服务器集群的日志数据，您可能会发现最佳选择是设置一个专用的归档 MongoDB 集群（参见下文“使用专用 MongoDB 集群归档所有日志数据”）。在 MongoDB 中备份和恢复 Message Logger 日志数据有多种方法。有关这些方法的更多信息，请参见 <http://docs.mongodb.org/v2.4/administration/backup/>。MongoDB 建议的方案包括：使用 MongoDB Management Service (MMS) 备份和恢复、使用文件系统快照备份和恢复、以及使用

MongoDB 工具备份和恢复。

MongoDB Management Service MMS

使用 MMS 备份意味着将数据备份到 MongoDB 云端，且不是免费的，因此不 ideal（理想）于大型生产部署。

此方法通过使用系统级工具（如 LVM）创建文件系统快照来备份所有 MongoDB 数据。该方法使用系统级工具创建持有 MongoDB 数据文件的设备的副本。这些方法速度快且可靠，但需要在 MongoDB 之外进行系统配置。此方案可用于备份单个 MongoDB 实例或集群。但是，备份整个集群需要复杂的操作，因此可能不适合执行定期的集群备份。有关如何使用文件系统快照进行备份和恢复的详细信息，请参见：Backup and Restore with Filesystem Snapshots、Backup a Sharded Cluster with Filesystem Snapshots。

MongoDB

此方法涉及使用 MongoDB 的 dump 和 restore 工具 mongodump 和 mongorestore 来备份和恢复日志数据。此方案可用于备份单个 MongoDB 实例或集群。执行备份时，可以通过提供查询选项来选择备份一个集合、一个数据库甚至集合的一部分。但是，此方案可能会对 MongoDB 性能产生不利影响，因为需要从 MongoDB 服务器读取数据。使用查询仅备份集合的一部分可能会加剧性能问题，因为 MongoDB 需要查询集合并将结果保存在内存中，如果 dump 的数据集很大，将消耗大量内存。总体而言，此方案不适合大型数据集，且不应在游戏服务器高峰时段执行。有关更多信息，请参见 Back Up and Restore with MongoDB Tools、Backup a Small Sharded Cluster with mongodump、Backup a Sharded Cluster with Database Dumps。

归档日志数据的其他方法

使用 mlcat.py 归档部分日志

使用专用 MongoDB 集群归档所有日志数据

如上所述，MongoDB

推荐的方法可能不适合定期备份/归档大型集群或大型数据集。一种替代方案是使用命令行工具 mlcat.py 来部分归档日志数据。使用 mlcat.py，您可以查询存储在 MongoDB 中的日志，并将输出重定向到文件以进行归档。这非常适合归档单个用户的日志（数据量不大）。有关如何使用 mlcat.py 的更多信息，请参见 http://docs/2/current/html/server_operations_guide/server_operations_guide.html#xref_Command_Line_Uutilities。请注意，此方法可能会影响 MongoDB 服务器性能，因为需要执行查询以从 MongoDB 读取数据。

归档大型服务器集群中的所有日志数据可能会产生海量数据。一种可能的方案是设置一个大型专用 MongoDB 集群，仅用于归档，并启动专用的 Message Logger 实例来接收来自服务器集群的日志，并将数据写入此 MongoDB 集群。

附录 H. 安装和配置 Carbon 与 Graphite

Carbon 和 Graphite 前置条件

附录 H. 安装和配置

Carbon 与 Graphite

安装 Carbon / Graphite、配置 Carbon、配置 WebConsole、WebConsole 分析。

Graphite/Carbon

是一种广泛使用的指标记录和查询服务，它将成为最终替代现有基于关系数据库的 StatLogger 版本的基础。此版本中包含了一个基于 Carbon 的 StatLogger 原型，以及 WebConsole 中对应的基于 Graphite 的 Graphs 模块，作为开发者预览版。



Carbon / Graphite 安装目前仅在 CentOS 6 主机上获得官方支持，但 StatLogger 和 WebConsole 仍可安装在 CentOS 5 上，并与 Carbon / Graphite 安装进行通信。本附录涵盖当 StatLogger 和 WebConsole 的 Graphs 模块已配置为使用 Carbon 数据存储时所需的 Carbon 和 Graphite 服务的安装与配置。有关如何配置 StatLogger 以利用此功能的更多信息，请参见“配置 StatLogger”。

StatLogger 的 Carbon 集成要求在与 StatLogger 实例相同的网络中安装并运行 Carbon 服务器。目前唯一官方支持的 Carbon 安装是通过 EPEL 仓库在 CentOS 6 上进行。本节假设您已经安装了一台运行 CentOS 6 发行版的新主机，并使用与“安装 EPEL 仓库”一节中所述类似的方法安装了 EPEL 仓库（在原来指定 CentOS 5 目录位置处替换为 CentOS 6 的目录位置）。



安装 Carbon / Graphite



根据内部测试，我们强烈建议将 Carbon 服务写入的目录（默认为 /var/lib/carbon）挂载到物理连接的磁盘上，因为已证明这比网络挂载或虚拟化磁盘能提供更好的查询性能。新的基于 Carbon 的 WebConsole Graphs 模块所使用的 Graphite Web 服务还需要一个 Web 服务器（Apache）。Carbon 和 Graphite 服务必须位于同一台物理机器上，以便 Graphite 能够读取从 Carbon 记录的数据。

在安装 Carbon 和 Graphite 之前，应在 Linux 服务器上禁用 SELinux。这可以通过编辑 /etc/selinux/config 并将：

```
SELINUX=enforcing
```

更改为：

```
SELINUX=disabled
```

此更改后重新启动 Linux 服务器。要在您的 CentOS 6 主机上安装 Carbon 和 Graphite，请以 root 身份执行以下命令：

```
# yum install graphite-web python-carbon
```

这将下载并安装 Carbon 数据记录服务器以及 Graphite 前端 Web 服务。要完成安装，请以 root 身份运行以下命令：

```
# cd /usr/lib/python2.6/site-packages/graphite
# su -s /bin/bash apache -c "python ./manage.py syncdb"
```

配置 Carbon

打开文件 `/etc/graphite-web/local_settings.py` 并编辑 `TIMEZONE` 值以与您当地的时区对应，遵循 TZ 时区格式。

接下来，必须配置 Apache 以允许跨站点请求。在文件 `/etc/httpd/conf.d/graphite-web.conf` 中，将以下行添加到 `VirtualHost` 部分：

```
Header set Access-Control-Allow-Origin "*"
```

或者，将星号替换为将用于提供 Carbon 版 Graphs 模块的 WebConsole 实例的基础 URI。有关 `Access-Control-Allow-Origin` 标头和跨站点资源访问的更多详细信息，请参见 https://developer.mozilla.org/en/docs/HTTP/Access_control_CORS 和/或 W3C 建议。

最后，必须（重新）启动 Apache Web 服务器：

```
# /etc/init.d/httpd restart
```

与基于数据库的 StatLogger 版本不同，Carbon 实现了自己的机制，随时间将指标聚合到较低分辨率的归档中。因此，Carbon 的聚合级别必须独立配置。

在 Graphite/Carbon 主机上，打开文件 `/etc/carbon/storage-schemas.conf`，并在 `[carbon]` 标头注释之后、第一个条目之前直接添加以下行：

```
[stat_logger]
pattern = ^stat_logger\.
retentions = 2s:1d,20s:2d,5m:30d,1h:2y
```



`[stat_logger]` 部分必须出现在可能匹配 StatLogger 指标名称的任何/所有其他部分之前，这一点至关重要，因为 Carbon 会应用第一个匹配的聚合规则。

同样在 Graphite/Carbon 主机上，打开或创建文件 `/etc/carbon/storage-aggregation.conf` 并添加以下行：

```
# Apply min when aggregating statistics containing the text "Min"
[stat_logger_min]
pattern = ^stat_logger\..+[_\.]Min[\w-]*$
aggregationMethod = min
xFilesFactor = 0.0

# Apply max when aggregating statistics containing the text "Max"
[stat_logger_max]
```

```

pattern = ^stat_logger\..+[_\.]Max[\w-]*$
aggregationMethod = max
xFilesFactor = 0.0

# Else apply average (default)
[stat_logger]
pattern = ^stat_logger
xFilesFactor = 0.0

```

有关每行含义的具体信息可在

<http://graphite.readthedocs.org/en/latest/config-carbon.html> 找到。

分配给 stat_logger 的前缀 pattern 必须与 StatLogger 的 preferences.xml 中 <carbon> 部分里的 <prefix> 标签的值相同，如“配置”中所述。

retentions 配置也必须与 preferences.xml 中的 <aggregation> 部分匹配。但请注意，preferences.xml 以样本数量声明聚合窗口，而 Carbon 以时间段声明聚合窗口。

Carbon 使用的一般模式是将分辨率级别声明为 “<一个点的分辨率>:<级别持续时间>”；例如，上面给出的配置片段声明了 4 个分辨率级别：最高为 2 秒分辨率的一天数据，最低为 1 小时分辨率的一年数据。

storage-aggregation.conf 中的配置告诉 Carbon，对于匹配文本 “Min” 或 “Max” 的统计信息，应分别按数学最小值或最大值进行聚合，而不是默认的平均值方法。

配置完成后，启动 Carbon 服务：

```
# /etc/init.d/carbon-cache start
```

配置 WebConsole

WebConsole 分析

必须显式启用 WebConsole 的 Carbon 版 Graphs 模块，并将其配置为访问相应的 Graphite/Carbon 主机。在已安装 BigWorld 服务器工具的机器上，位于 /etc/bigworld/web_console.conf 的 WebConsole 配置文件中，找到以下行：

```
web_console.graphs.on = False
```

并将其更改为：

```
web_console.graphs.on = True
```

最后，将 Graphite/Carbon 主机名和端口设置为您刚安装 Graphite/Carbon 的机器：

```
web_console.graphs.graphite_host = 'http://your_graphite_server:80'
```

这些配置更改后需要重新启动 WebConsole 才能生效：

如果 WebConsole 安装在 CentOS 5 上：

```
# /etc/init.d/bw_web_console start
```


如果 WebConsole 安装在 CentOS 7 上:

```
# systemctl start bw_web_console
```

Carbon 版的 Graphs 模块现在将出现在 WebConsole 左侧菜单中。

WebConsole 包含将分析数据（如页面请求时间、CPU 负载和内存使用情况）发送到 Carbon 的能力。然后可以在 Graphite 中查看这些数据。要配置 WebConsole 分析，首先配置一个 Carbon 服务器。有关如何配置 Carbon 和 Graphite 的信息，请参见附录 H“安装和配置 Carbon 和 Graphite”。

默认的 Carbon 聚合间隔为 1 分钟。除非在您的 Carbon 配置中另有指定，数据点将以标准频率进行聚合和存储。我们建议更改存储 schema 以提高粒度。

在 Graphite/Carbon 主机上，打开文件 `/etc/carbon/storage-schemas.conf`，并在 `[carbon]` 第一个条目之前插入一个包含所需间隔的部分。例如：

```
[web_console]
pattern = ^web_console\.
retentions = 15s:1d,60s:30d,1h:1y
```

有关在 Carbon 中指定 `retentions` 的信息，请参见 Carbon 文档。更改 Carbon 配置后，重新启动 Carbon 服务。

接下来，在 WebConsole 配置文件中，启用分析并指定您的 Carbon 服务器。从命令行运行时，WebConsole 使用配置文件 `dev.cfg`。当作为从 RPM 安装的系统守护进程运行时，它使用 `/etc/bigworld/web_console.conf` 配置文件。

```
web_console.analytics.on = True
web_console.analytics.carbon_host = "your_carbon_server"
web_console.analytics.carbon_port = 2004
web_console.analytics.stat_cache_timeout = 1 # seconds
```

默认的 Carbon 端口为 2004。如果您的 Carbon 服务已配置为使用备用端口，请按需更改上述端口号。为减少不必要的磁盘访问，WebConsole 将按 `web_console.analytics.stat_cache_timeout` 指定的间隔从 `/proc` 收集并缓存 CPU 和内存使用统计信息。虽然默认值 1 秒应该足以减少对 `/proc` 的不必要磁盘访问，但如有必要可以增大此值。此值不应增大到超过 Carbon 存储 schema 中指定的最短 Carbon 保留间隔（在上面的示例中为 15 秒）。

更改 WebConsole 配置后，重新启动 WebConsole。

附录 I. 故障排除

检查 BWMachined 是否正在运行

BWMachined 故障排除

StatLogger、BigWorld

支持。虽然我们努力防止在安装过程中出现问题，但一些问题仍然不可避免。本附录旨在概述安装 BigWorld 服务器和工具时可能发生的一些较常见的故障情况。

要检查守护进程是否正在运行，请使用 `control_cluster.py` 工具。有关详细信息，请参见 `Server Operations Guide` 中的 “`Server Command-Line Utilities`” 一节。要使用 `control_cluster.py` 检查 `BWMachined` 的状态，请发出以下命令：

```
$ game/tools/bigworld/server/control_cluster.py cinfo
```

正确运行 `BWMachined` 的机器应该会显示出来，如下例所示：

```
shire 10.40.3.37
| 0 processes | 0%, 0% of 2000MHz ( |
4% mem)
```

确保进程具有一个位于内部网络上的地址。如果没有，请确保您的广播路由设置正确。如果您的机器未列出，但 `BWMachined` 正在运行，则可能需要检查防火墙规则。有关更多详细信息，请参见本文档中的“安全”一节。您还可以运行以下命令来检查集群中 `bwmachined` 进程之间的关系。

```
$ game/tools/bigworld/server/control_cluster.py checkring
```

StatLogger

安装 `BigWorld` 服务时可能出现的一些最常见问题，是由于初始安装后 `BWMachined` 未运行所致。以下概述了遇到的最常见问题以及如何解决这些问题。应按顺序检查这些步骤。

`BWMachined` 是否正在运行？

```
/sbin/service bwmachined2 status
```

如果 `BWMachined` 正在运行，您可能需要调整防火墙规则，以允许 `UDP` 广播消息在服务器集群中发送和接收。有关更多信息，请参见“安全”一节。如果 `BWMachined` 未运行，请继续执行以下步骤。

`BWMachined` 是否能够启动？

`BWMachined` 通常能够成功启动，但在尝试向网络发送广播消息时会失败。为了确定启动时发生了什么，我们需要检查系统日志（`syslog`），默认可在 `/var/log/messages` 中找到。成功启动的输出应如下所示：

```
/opt/bigworld/2.1/bwmachined/sbin/bwmachined2: --- BWMachined start ---
/opt/bigworld/2.1/bwmachined/sbin/bwmachined2: Host architecture: 64 bit.
/opt/bigworld/2.1/bwmachined/sbin/bwmachined2: Using gettime timing
/opt/bigworld/2.1/bwmachined/sbin/bwmachined2: Broadcast discovery receipt from 10.40.3.145.
/opt/bigworld/2.1/bwmachined/sbin/bwmachined2: Confirmed 10.40.3.145 (eth0) as default broadcast ro
```

这里的关键行是确认默认广播路由的最后一行。如果您没有看到此行，而是看到错误消息，则很可能需要调整默认广播路由规则。有关更多详细信息，请参见“路由”一节。如果您在系统日志中难以找到 `BWMachined` 输出，请尝试重新启动 `BWMachined`，它启动时应生成新的日志。

`StatLogger` 遇到的一些最常见问题包括：